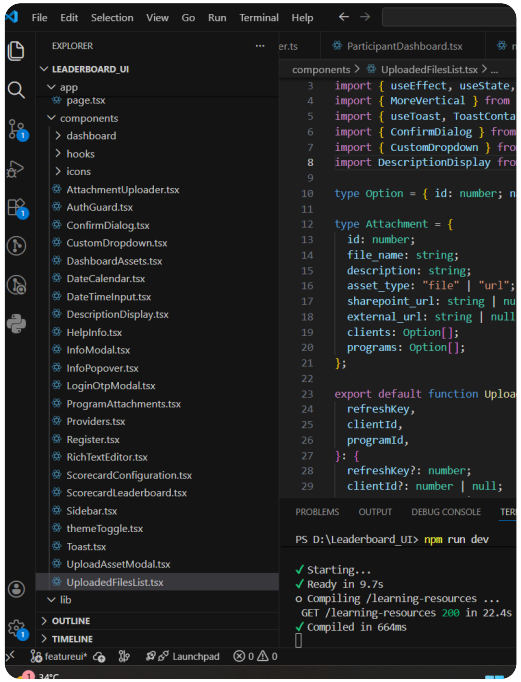

this is my frontend component section

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";

type Option = { id: number; name: string };
```

```
type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: Option[];
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {

  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [clients, setClients] = useState<Option[]>([]);
  const [draftAssignments, setDraftAssignments] = useState<any>({});
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [filteredPrograms, setFilteredPrograms] =
useState<Record<number, Option[]>>({});
```

```
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const { toasts, showToast, removeToast } = useToast();

  async function fetchFilesREST() {
    let url = `${API}/api/attachments/`;

    if (clientId || programId) {
      const params = new URLSearchParams();
      if (clientId) params.append("client", String(clientId));
      if (programId) params.append("program", String(programId));
      url += `?${params.toString()}`;
    }

    const res = await fetch(url);
    setFiles(await res.json());
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey, clientId, programId]);

  async function fetchClientsGQL() {
    const query = `query { clients { id name } }`;

    const res = await fetch(`${API}/api/attachments/graphql/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({ query }),
    });

    const json = await res.json();
    setClients(json.data.clients);
  }

  useEffect(() => {
    fetchClientsGQL();
  }, []);

  async function fetchProgramsByClientGQL(fileId: number, clientId:
number) {
    const query =
      query($clientId: Int!) {
        programsByClient(clientId: $clientId) {
          id
          name
        }
      }
    ;

    const res = await fetch(${API}/api/attachments/graphql/, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query, variables: { clientId } }),
    });

    const json = await res.json();
```

```
    setFilteredPrograms(prev => ({
      ...prev,
      [fileId]: json.data.programsByClient,
    }));
  }

  async function saveAssignmentREST(fileId: number) {
    const data = draftAssignments[fileId];

    if (!data?.clients?.length || !data?.programs?.length) {
      showToast("Select Client and Program", "error");
      return;
    }

    await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });

    fetchFilesREST();
    // Reset draft selection for this file so dropdowns start blank next
time
    setDraftAssignments((prev: any) => {
      const next = { ...prev };
      delete next[fileId];
      return next;
    });
```

```
  setOpenMenu(null);
}

function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;
  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles(prev => prev.filter(f => f.id !== fileId));
    showToast("File deleted successfully", "success");
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
```

```
}

function updateDraft(fileId: number, field: string, values: number[]) {
  setDraftAssignments((prev: any) => ({
    ...prev,
    [fileId]: { ...prev[fileId], [field]: values },
  }));
}

const filteredFiles = files.filter(f =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

const unassigned = filteredFiles.filter(
  f => f.clients.length === 0 && f.programs.length === 0
);

const assigned = filteredFiles.filter(
  f => f.clients.length > 0 || f.programs.length > 0
);

function FileCard(file: Attachment) {

  const cardRef = useRef<HTMLDivElement>(null);
  const draft = draftAssignments[file.id] || {};
  const disableSave = !draft.clients?.length || !draft.programs?.length;

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
```

```
      const target = event.target as HTMLElement | null;
      // If click is inside any CustomDropdown overlay, don't close the
assignment popup
      if (target && target.closest(".custom-dropdown-overlay")) {
        return;
      }
      if (
        openMenu === file.id &&
        cardRef.current &&
        !cardRef.current.contains(event.target as Node)
      ) {
        setOpenMenu(null);
      }
    }

    document.addEventListener("mousedown", handleClickOutside);
    return () => {
      document.removeEventListener("mousedown",
handleClickOutside);
    };
  }, [openMenu]);

  return (
    <div
      ref={cardRef}
      className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
    >
```

```
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() =>
            setOpenMenu(openMenu === file.id ? null : file.id)
          }
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold">{file.file_name}</p>
        {file.description && <DescriptionDisplay html={file.description}
className="text-xs text-slate-500 dark:text-slate-400" />}

        <p className="text-xs">
          Clients: {file.clients.map(c => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map(p => p.name).join(", ") || "None"}
        </p>

        {(file.asset_type === "file" && file.sharepoint_url) ||
          (file.asset_type === "url" && file.external_url) ? (
          <a
            href={
              file.asset_type === "file"
                ? file.sharepoint_url!
```

```
              : file.external_url!
            }
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View {file.asset_type === "file" ? "File" : "Link"}
          </a>
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}

        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[260px]
              bg-white dark:bg-slate-900
              border border-slate-200 dark:border-slate-700
              p-3 rounded-lg shadow-lg
              space-y-2 z-50
            "
          >

            <CustomDropdown
              value={String(draft.clients?.[0] || "")}
```

```
            onChange={(selectedValue) => {
              const cid = Number(selectedValue);
              updateDraft(file.id, "clients", [cid]);
              updateDraft(file.id, "programs", []);
              fetchProgramsByClientGQL(file.id, cid);
            }}
            options={clients.map((c) => ({ id: String(c.id), label: c.name }))}
            placeholder="Select Client"
            shouldSort={true}
            searchable={clients.length > 5}
            className="w-full text-xs"
          />

          <CustomDropdown
            value={String(draft.programs?.[0] || "")}
            onChange={(selectedValue) =>
              updateDraft(file.id, "programs", [Number(selectedValue)])
            }
            options={(filteredPrograms[file.id] || []).map((p) => ({
              id: String(p.id),
              label: p.name,
            }))}
            placeholder="Select Program"
            shouldSort={true}
            className="w-full text-xs"
            searchable={(filteredPrograms[file.id] || []).length > 5}
          />

          <div className="flex gap-2 pt-1">
```

```
            <button
              onClick={() => saveAssignmentREST(file.id)}
              disabled={disableSave}
              className="flex-1 px-2 py-1 text-xs rounded bg-emerald-
600 text-white disabled:opacity-50"
            >
              Save
            </button>

            <button
              onClick={() => requestDeleteFile(file.id)}
              className="flex-1 px-2 py-1 text-xs rounded bg-red-600
text-white"
            >
              Delete
            </button>
          </div>
        </div>
      )}
    </div>
  );
}

return (
  <div className="space-y-6">
      <div className="mb-4 flex justify-center">
    <div className="relative w-full max-w-md">
      <svg
        className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
```

```
text-slate-400 dark:text-slate-500"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
          />
        </svg>
        <input
          type="text"
          placeholder="Search files..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
        />
      </div>
    </div>
      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
```

```
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">

        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100">📂
UnAssigned Files</h3>
          <div className="space-y-3 h-[500px] overflow-y-auto">
            {unassigned.length > 0 ? (
              unassigned.map(file => (
                <FileCard key={file.id} {...file} />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                No unassigned files match your search
              </div>
            )}
          </div>
        </div>

        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
```

```
border border-slate-200 dark:border-slate-700">
        <h3 className="mb-3 text-slate-900 dark:text-slate-100">✅
Assigned Files</h3>
        <div className="space-y-3 h-[500px] overflow-y-auto">
         {assigned.length > 0 ? (
           assigned.map(file => (
             <FileCard key={file.id} {...file} />
           ))
         ) : (
           <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
             No assigned files match your search
           </div>
         )}
        </div>
       </div>

      </div>
    </div>
  );
}
```

this is code for uploadedfileslist.tsx
for video files i want to build a custom videoplayer for videos in learning resources tell me to do it and check step by step with propar path

Yes — **best approach is to build a reusable custom video player component** and then use it inside `UploadedFilesList.tsx` only for video files in Learning Resources.

Since your structure is clear from screenshot, I'll give you the **proper file paths** and **step-by-step integration**.

✅ Goal

In **Learning Resources**:

- If attachment is a **normal file (pdf/doc/etc.)** → keep **View File**
- If attachment is a **video file (mp4/webm/mov etc.)** → show **custom video player**
- If attachment is a **URL** → keep **View Link**

📁 Proper file paths in your project

Based on your screenshot:

- Existing list component:
  - `components/UploadedFilesList.tsx`
- Best place for new video player:
  - `components/CustomVideoPlayer.tsx`

So we will create:

```
LEADERBOARD_UI/
└── components/
```

Bash

✅ Step 1: Create a reusable custom video player component

Create file:

Path:

`components/CustomVideoPlayer.tsx`

Code:

```
"use client";

import { useRef, useState, useEffect } from "react";
import { Play, Pause, Volume2, VolumeX, Maximize, Minimize } from "lucide-react";

type CustomVideoPlayerProps = {
  src: string;
  fileName?: string;
};

export default function CustomVideoPlayer({
  src,
  fileName,
}: CustomVideoPlayerProps) {
  const videoRef = useRef<HTMLVideoElement>(null);
  const containerRef = useRef<HTMLDivElement>(null);
```

```
  const [isPlaying, setIsPlaying] = useState(false);
  const [isMuted, setIsMuted] = useState(false);
  const [progress, setProgress] = useState(0);
  const [duration, setDuration] = useState(0);
  const [isFullscreen, setIsFullscreen] = useState(false);

  const togglePlay = () => {
    if (!videoRef.current) return;

    if (videoRef.current.paused) {
      videoRef.current.play();
      setIsPlaying(true);
    } else {
      videoRef.current.pause();
      setIsPlaying(false);
    }
  };

  const toggleMute = () => {
    if (!videoRef.current) return;

    videoRef.current.muted = !videoRef.current.muted;
    setIsMuted(videoRef.current.muted);
  };

  const handleTimeUpdate = () => {
    if (!videoRef.current) return;

    const current = videoRef.current.currentTime;
    const total = videoRef.current.duration || 0;

    setProgress(total ? (current / total) * 100 : 0);
  };

  const handleLoadedMetadata = () => {
```

```
    if (!videoRef.current) return;
    setDuration(videoRef.current.duration || 0);
  };

  const handleSeek = (e: React.ChangeEvent<HTMLInputElement>) => {
    if (!videoRef.current) return;

    const newProgress = Number(e.target.value);
    const total = videoRef.current.duration || 0;
    videoRef.current.currentTime = (newProgress / 100) * total;
    setProgress(newProgress);
  };

  const toggleFullscreen = async () => {
    if (!containerRef.current) return;

    if (!document.fullscreenElement) {
      await containerRef.current.requestFullscreen();
      setIsFullscreen(true);
    } else {
      await document.exitFullscreen();
      setIsFullscreen(false);
    }
  };

  useEffect(() => {
    const handleFullscreenChange = () => {
      setIsFullscreen(!!document.fullscreenElement);
    };

    document.addEventListener("fullscreenchange", handleFullscreenChange);
    return () =>
      document.removeEventListener("fullscreenchange", handleFullscreenChange);
  }, []);
```

```
  const formatTime = (time: number) => {
    if (!time || Number.isNaN(time)) return "0:00";

    const mins = Math.floor(time / 60);
    const secs = Math.floor(time % 60);

    return `${mins}:${secs.toString().padStart(2, "0")}`;
  };

  return (
    <div
      ref={containerRef}
      className="mt-2 rounded-xl overflow-hidden border border-slate-200 dark:border-slate-700 bg
    >
      <video
        ref={videoRef}
        src={src}
        className="w-full h-auto max-h-[280px] bg-black"
        onTimeUpdate={handleTimeUpdate}
        onLoadedMetadata={handleLoadedMetadata}
        onPlay={() => setIsPlaying(true)}
        onPause={() => setIsPlaying(false)}
        controls={false}
      />

      <div className="p-3 bg-slate-900 text-white space-y-2">
        {fileName && (
          <p className="text-xs text-slate-300 truncate">{fileName}</p>
        )}

        <input
          type="range"
          min="0"
          max="100"
          value={progress}
```

```
            onChange={handleSeek}
            className="w-full accent-emerald-500"
          />

          <div className="flex items-center justify-between gap-2">
            <div className="flex items-center gap-2">
              <button
                onClick={togglePlay}
                className="p-2 rounded-lg bg-emerald-600 hover:bg-emerald-700"
              >
                {isPlaying ? <Pause size={16} /> : <Play size={16} />}
              </button>

              <button
                onClick={toggleMute}
                className="p-2 rounded-lg bg-slate-700 hover:bg-slate-600"
              >
                {isMuted ? <VolumeX size={16} /> : <Volume2 size={16} />}
              </button>

              <span className="text-xs text-slate-300">
                {videoRef.current
                  ? `${formatTime(videoRef.current.currentTime)} / ${formatTime(duration)}`
                  : `0:00 / ${formatTime(duration)}`}
              </span>
            </div>

            <button
              onClick={toggleFullscreen}
              className="p-2 rounded-lg bg-slate-700 hover:bg-slate-600"
            >
              {isFullscreen ? <Minimize size={16} /> : <Maximize size={16} />}
            </button>
          </div>
        </div>
```

```
      </div>
    );
  }
```

✅ Step 2: Import this component in `UploadedFilesList.tsx`

File path:

`components/UploadedFilesList.tsx`

At the top, add:

TypeScript

```
import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import CustomVideoPlayer from "@/components/CustomVideoPlayer";
```

✅ Step 3: Detect whether file is a video

Right now your `Attachment` type does **not** contain mime type or extension from backend, so easiest frontend way is:

- Detect by file name extension
- Use `file.file_name`

Add helper function inside `UploadedFilesList.tsx`

Add this **inside component**, above `const filteredFiles = ...`

```
function isVideoFile(file: Attachment) {
  if (file.asset_type !== "file" || !file.file_name) return false;

  const lower = file.file_name.toLowerCase();

  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}
```

✅ Step 4: Replace "View File" section with custom video player logic

Right now you have this block inside `FileCard`:

```
{(file.asset_type === "file" && file.sharepoint_url) ||
  (file.asset_type === "url" && file.external_url) ? (
  <a
    href={
      file.asset_type === "file"
        ? file.sharepoint_url!
        : file.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
  >
    View {file.asset_type === "file" ? "File" : "Link"}
  </a>
) : (
  <span className="text-xs text-slate-400 dark:text-slate-500">
    No link available
  </span>
)}
```

🔁 Replace it with this:

```
{file.asset_type === "file" && file.sharepoint_url ? (
  isVideoFile(file) ? (
    <CustomVideoPlayer
```

```
            src={file.sharepoint_url}
            fileName={file.file_name}
          />
        ) : (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
          >
            View File
          </a>
        )
      ) : file.asset_type === "url" && file.external_url ? (
        <a
          href={file.external_url}
          target="_blank"
          rel="noopener noreferrer"
          className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
        >
          View Link
        </a>
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}
```

✅ Step 5: Important SharePoint issue you must check

This is **very important** for your case.

A custom HTML5 `<video>` player will work only if the `sharepoint_url` is:

- a **direct downloadable/streamable file URL**
- AND accessible in browser
- AND not forcing SharePoint preview page / auth redirect

If current `sharepoint_url` opens a SharePoint page like:

- preview page
- office viewer page
- redirect page

Then `<video src="sharepoint_url">` may **not play**

For video player to work best, backend should return a direct raw file URL

In backend, for SharePoint/Graph uploads, the best is to store:

- `@microsoft.graph.downloadUrl`
  OR
- a direct file content endpoint

Example Graph content endpoint format:

```
https://graph.microsoft.com/v1.0/sites/{site-id}/drives/{drive-id}/items/{item-id}/content
```

⚠️ But this Graph content URL needs auth header, so browser `<video>` cannot easily use it directly unless you proxy it through backend.

✅ Recommended best architecture for your project

Since you already have Django backend, **best professional approach** is:

Backend:

Create a new endpoint like:

```bash
</> Bash
```

That endpoint:

- checks attachment
- gets SharePoint access token
- fetches file content from Graph/SharePoint
- returns streamed response to frontend

Frontend:

Use:

```typescript
</> TypeScript
```

This is **much better** than directly using SharePoint URL.

✅ Step 6 (Best version): Use backend stream endpoint instead of `sharepoint_url`

If you want robust implementation, use this in frontend:

Replace:

```
<CustomVideoPlayer
  src={file.sharepoint_url}
  fileName={file.file_name}
/>
```

TypeScript

With:

```
<CustomVideoPlayer
  src={`${API}/api/attachments/${file.id}/stream/`}
  fileName={file.file_name}
/>
```

TypeScript

✅ Step 7: What backend endpoint you should create (recommended)

Since you asked for proper path, for your Django backend (attachments app), create:

API path:

Bash

Example Django URL path

In your attachments app `urls.py`:

Python Run

Example Django view (recommended pattern)

```python
from django.http import StreamingHttpResponse, JsonResponse
from .models import Attachment
import requests

def stream_attachment(request, pk):
    try:
        attachment = Attachment.objects.get(pk=pk)
    except Attachment.DoesNotExist:
        return JsonResponse({"error": "Attachment not found"}, status=404)

    if not attachment.sharepoint_item_id:
        return JsonResponse({"error": "No SharePoint file found"}, status=404)
```

```
    access_token = get_access_token()

    # Example: build your graph content URL using stored site/drive/item ids
    graph_url = f"https://graph.microsoft.com/v1.0/sites/{SITE_ID}/drives/{DRIVE_ID}/items/{attac

    resp = requests.get(
        graph_url,
        headers={"Authorization": f"Bearer {access_token}"},
        stream=True,
    )

    if resp.status_code != 200:
        return JsonResponse({"error": "Unable to stream video"}, status=resp.status_code)

    content_type = resp.headers.get("Content-Type", "application/octet-stream")

    response = StreamingHttpResponse(
        resp.iter_content(chunk_size=8192),
        content_type=content_type,
    )
    response["Content-Disposition"] = f'inline; filename="{attachment.file_name}"'
    return response
```

⚠️ Very important improvement for video seeking (must do later)

For custom video player seeking (`drag progress bar`) to work smoothly, backend should support **Range Requests**.

That means:

- browser sends header:

  - `Range: bytes=...`
- backend forwards it to Graph
- backend returns:
  - `206 Partial Content`

Without this:

- video may still play
- but seeking/jumping may be poor or broken

So **Phase 1**: get video playing first

Then **Phase 2**: add range support

✅ Best step-by-step order for you (recommended)

Because you're already debugging SharePoint flows, do this in this order:

Phase 1 (Frontend only quick test)

1. Create `components/CustomVideoPlayer.tsx`
2. Import it in `components/UploadedFilesList.tsx`
3. Add `isVideoFile()` helper
4. Replace "View File" block with video player rendering
5. Test with existing `sharepoint_url`

If it works:

✅ Done for now

If it does NOT work:

That means SharePoint URL is not directly streamable

Then do:

Phase 2 (Proper production solution)

6. Create backend endpoint:
   - `/api/attachments/<id>/stream/`
7. Return file stream from Django using Graph token
8. Update frontend player src to:
   - `${API}/api/attachments/${file.id}/stream/`
9. Test playback
10. Later add **Range header support** for seek support

✅ Final exact changes you need in frontend

---

1. New file

`components/CustomVideoPlayer.tsx`

(create with full code above)

---

2. Update imports in

`components/UploadedFilesList.tsx`

TypeScript

---

3. Add helper inside component

```typescript
function isVideoFile(file: Attachment) {
  if (file.asset_type !== "file" || !file.file_name) return false;

  const lower = file.file_name.toLowerCase();

  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}
```

---

4. Replace file/link render block in `FileCard`

Use:

```
          {file.asset_type === "file" && file.sharepoint_url ? (
            isVideoFile(file) ? (
              <CustomVideoPlayer
                src={`${API}/api/attachments/${file.id}/stream/`}
                fileName={file.file_name}
              />
            ) : (
              <a
                href={file.sharepoint_url}
                target="_blank"
                rel="noopener noreferrer"
                className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
              >
                View File
              </a>
            )
          ) : file.asset_type === "url" && file.external_url ? (
            <a
              href={file.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
            >
              View Link
            </a>
          ) : (
            <span className="text-xs text-slate-400 dark:text-slate-500">
              No link available
            </span>
          )}
```

🔥 My strong recommendation for your project

Since your project already uses:

- Django backend
- SharePoint / Graph upload
- stored DB records

👉 **Do NOT rely on raw SharePoint URLs for video playback long-term.**
Use a backend streaming endpoint.

That will make:

- auth handling easier
- external user issues easier
- consistent behavior
- future secure access control possible

✅ Next best step for you

If you want, I can now give you the **exact Django backend code for** `/api/attachments/<id>/stream/` based on your existing SharePoint upload/delete setup (same app structure as your attachments APIs), with:

- proper `urls.py`
- proper `views.py`
- using your existing `get_access_token()`
- using your existing attachment model fields
- and then I'll give you the **final exact updated** `UploadedFilesList.tsx` **full code** ready to paste.

👉 If you want, send me these 3 backend files next and I'll wire it exactly to your project:

1. `attachments/models.py`
2. `attachments/views.py`
3. `attachments/urls.py`

Then I'll give you **copy-paste ready final code** for both frontend + backend.

i have made the changes but video is not playing i want you to design me good proparlly working customvideopage.tsx

Yes — I can give you a **proper, cleaner, better custom video player component** that is production-friendly for your UI.
But **very important** first:

⚠️ Important truth

If your video is **not playing at all**, then **the problem is usually NOT the player design**.

A custom player can only control a `<video>` tag.
If this fails:

TypeScript

then **no custom UI can make it play**.

For SharePoint-based files, this usually happens because:

- `sharepoint_url` is a **preview page URL**, not raw video file
- SharePoint requires **auth / cookies / redirect**
- The URL returns **HTML page** instead of video bytes
- CORS / access issue
- No proper streamable endpoint

👉 So I'll give you:

1. **A much better `CustomVideoPlayer.tsx`**
2. **A quick test to confirm whether URL is streamable**
3. **What backend endpoint you should use for guaranteed working playback**

✅ Step 1: Replace your current player with a better one

File path:

`components/CustomVideoPlayer.tsx`

Full code (copy-paste)

```
"use client";

import { useEffect, useRef, useState } from "react";
import {
```

```
  Play,
  Pause,
  Volume2,
  VolumeX,
  Maximize,
  Minimize,
  RotateCcw,
} from "lucide-react";

type CustomVideoPlayerProps = {
  src: string;
  fileName?: string;
  poster?: string;
  autoPlay?: boolean;
};

export default function CustomVideoPlayer({
  src,
  fileName,
  poster,
  autoPlay = false,
}: CustomVideoPlayerProps) {
  const videoRef = useRef<HTMLVideoElement>(null);
  const containerRef = useRef<HTMLDivElement>(null);
  const controlsTimeoutRef = useRef<NodeJS.Timeout | null>(null);

  const [isReady, setIsReady] = useState(false);
  const [isPlaying, setIsPlaying] = useState(false);
  const [isMuted, setIsMuted] = useState(false);
  const [volume, setVolume] = useState(1);
  const [currentTime, setCurrentTime] = useState(0);
  const [duration, setDuration] = useState(0);
  const [progress, setProgress] = useState(0);
  const [showControls, setShowControls] = useState(true);
  const [isFullscreen, setIsFullscreen] = useState(false);
```

```
  const [hasEnded, setHasEnded] = useState(false);
  const [errorMessage, setErrorMessage] = useState("");

  useEffect(() => {
    return () => {
      if (controlsTimeoutRef.current) clearTimeout(controlsTimeoutRef.current);
    };
  }, []);

  useEffect(() => {
    const handleFullscreenChange = () => {
      setIsFullscreen(!!document.fullscreenElement);
    };

    document.addEventListener("fullscreenchange", handleFullscreenChange);
    return () => {
      document.removeEventListener("fullscreenchange", handleFullscreenChange);
    };
  }, []);

  const showControlsTemporarily = () => {
    setShowControls(true);

    if (controlsTimeoutRef.current) clearTimeout(controlsTimeoutRef.current);

    if (videoRef.current && !videoRef.current.paused) {
      controlsTimeoutRef.current = setTimeout(() => {
        setShowControls(false);
      }, 2500);
    }
  };

  const togglePlay = async () => {
    const video = videoRef.current;
    if (!video) return;
```

```
    try {
      if (video.paused || video.ended) {
        setHasEnded(false);
        await video.play();
      } else {
        video.pause();
      }
    } catch (err) {
      console.error("Video play failed:", err);
      setErrorMessage("Unable to play this video. The source may not be streamable.");
    }
  };

  const restartVideo = async () => {
    const video = videoRef.current;
    if (!video) return;

    try {
      video.currentTime = 0;
      setHasEnded(false);
      await video.play();
    } catch (err) {
      console.error("Video restart failed:", err);
    }
  };

  const toggleMute = () => {
    const video = videoRef.current;
    if (!video) return;

    const nextMuted = !video.muted;
    video.muted = nextMuted;
    setIsMuted(nextMuted);
  };
```

```
const handleVolumeChange = (value: number) => {
  const video = videoRef.current;
  if (!video) return;

  const safeVolume = Math.max(0, Math.min(1, value));
  video.volume = safeVolume;
  video.muted = safeVolume === 0;

  setVolume(safeVolume);
  setIsMuted(safeVolume === 0);
};

const handleSeek = (value: number) => {
  const video = videoRef.current;
  if (!video || !duration) return;

  const newTime = (value / 100) * duration;
  video.currentTime = newTime;
  setCurrentTime(newTime);
  setProgress(value);
};

const toggleFullscreen = async () => {
  if (!containerRef.current) return;

  try {
    if (!document.fullscreenElement) {
      await containerRef.current.requestFullscreen();
    } else {
      await document.exitFullscreen();
    }
  } catch (err) {
    console.error("Fullscreen error:", err);
  }
```

```
  };

  const skipTime = (seconds: number) => {
    const video = videoRef.current;
    if (!video) return;

    const nextTime = Math.max(0, Math.min(video.currentTime + seconds, duration || video.duration
    video.currentTime = nextTime;
  };

  const formatTime = (time: number) => {
    if (!Number.isFinite(time) || time < 0) return "0:00";

    const hrs = Math.floor(time / 3600);
    const mins = Math.floor((time % 3600) / 60);
    const secs = Math.floor(time % 60);

    if (hrs > 0) {
      return `${hrs}:${String(mins).padStart(2, "0")}:${String(secs).padStart(2, "0")}`;
    }

    return `${mins}:${String(secs).padStart(2, "0")}`;
  };

  const onLoadedMetadata = () => {
    const video = videoRef.current;
    if (!video) return;

    setDuration(video.duration || 0);
    setIsReady(true);
    setErrorMessage("");

    if (autoPlay) {
      video.play().catch((err) => {
        console.error("Autoplay blocked:", err);
```

```
      });
    }
  };

  const onTimeUpdate = () => {
    const video = videoRef.current;
    if (!video) return;

    const current = video.currentTime || 0;
    const total = video.duration || 0;

    setCurrentTime(current);
    setProgress(total ? (current / total) * 100 : 0);
  };

  const onPlay = () => {
    setIsPlaying(true);
    setHasEnded(false);
    showControlsTemporarily();
  };

  const onPause = () => {
    setIsPlaying(false);
    setShowControls(true);
  };

  const onEnded = () => {
    setIsPlaying(false);
    setHasEnded(true);
    setShowControls(true);
  };

  const onError = () => {
    const video = videoRef.current;
    console.error("Video failed to load", {
```

```
      src,
      networkState: video?.networkState,
      readyState: video?.readyState,
      error: video?.error,
    });

    setErrorMessage(
      "This video could not be loaded. Usually this means the SharePoint URL is not a direct stre
    );
  };

  return (
    <div
      ref={containerRef}
      className="mt-3 overflow-hidden rounded-2xl border border-slate-200 dark:border-slate-700 b
      onMouseMove={showControlsTemporarily}
      onMouseLeave={() => {
        if (isPlaying) setShowControls(false);
      }}
    >
      <div className="relative group">
        <video
          ref={videoRef}
          src={src}
          poster={poster}
          preload="metadata"
          playsInline
          controls={false}
          className="w-full bg-black max-h-[360px]"
          onLoadedMetadata={onLoadedMetadata}
          onTimeUpdate={onTimeUpdate}
          onPlay={onPlay}
          onPause={onPause}
          onEnded={onEnded}
          onError={onError}
```

```
            onClick={togglePlay}
          />

          {/* Center Play/Replay Button */}
          <div className="absolute inset-0 flex items-center justify-center pointer-events-none">
            {(!isPlaying || hasEnded) && !errorMessage && (
              <button
                onClick={hasEnded ? restartVideo : togglePlay}
                className="pointer-events-auto flex items-center justify-center rounded-full bg-bla
              >
                {hasEnded ? <RotateCcw size={28} /> : <Play size={28} className="ml-1" />}
              </button>
            )}
          </div>

          {/* Error Overlay */}
          {errorMessage && (
            <div className="absolute inset-0 flex items-center justify-center bg-black/75 p-4">
              <div className="max-w-md text-center">
                <p className="text-sm text-red-300 font-medium mb-2">Video Playback Error</p>
                <p className="text-xs text-slate-200">{errorMessage}</p>
              </div>
            </div>
          )}

          {/* Loading Overlay */}
          {!isReady && !errorMessage && (
            <div className="absolute inset-0 flex items-center justify-center bg-black/40">
              <div className="text-white text-sm">Loading video...</div>
            </div>
          )}

          {/* Controls */}
          <div
            className={`absolute bottom-0 left-0 right-0 bg-gradient-to-t from-black/90 via-black/5
```

```
            showControls || !isPlaying ? "opacity-100" : "opacity-0"
          }`}
        >
          {/* Progress */}
          <input
            type="range"
            min={0}
            max={100}
            step={0.1}
            value={progress}
            onChange={(e) => handleSeek(Number(e.target.value))}
            className="w-full accent-emerald-500 cursor-pointer"
          />

          {/* Bottom controls */}
          <div className="mt-2 flex items-center justify-between gap-3">
            <div className="flex items-center gap-2 flex-wrap">
              <button
                onClick={togglePlay}
                className="p-2 rounded-lg bg-emerald-600 hover:bg-emerald-700 text-white transiti
              >
                {isPlaying ? <Pause size={16} /> : <Play size={16} />}
              </button>

              <button
                onClick={() => skipTime(-10)}
                className="px-2 py-1 text-xs rounded-lg bg-slate-700 hover:bg-slate-600 text-whit
              >
                -10s
              </button>

              <button
                onClick={() => skipTime(10)}
                className="px-2 py-1 text-xs rounded-lg bg-slate-700 hover:bg-slate-600 text-whit
              >
```

```
            +10s
          </button>

          <button
            onClick={toggleMute}
            className="p-2 rounded-lg bg-slate-700 hover:bg-slate-600 text-white"
          >
            {isMuted ? <VolumeX size={16} /> : <Volume2 size={16} />}
          </button>

          <input
            type="range"
            min={0}
            max={1}
            step={0.05}
            value={isMuted ? 0 : volume}
            onChange={(e) => handleVolumeChange(Number(e.target.value))}
            className="w-20 accent-emerald-500 cursor-pointer"
          />

          <span className="text-xs text-slate-200 whitespace-nowrap">
            {formatTime(currentTime)} / {formatTime(duration)}
          </span>
        </div>

        <button
          onClick={toggleFullscreen}
          className="p-2 rounded-lg bg-slate-700 hover:bg-slate-600 text-white transition"
        >
          {isFullscreen ? <Minimize size={16} /> : <Maximize size={16} />}
        </button>
      </div>

      {fileName && (
        <p className="mt-2 text-[11px] text-slate-300 truncate">{fileName}</p>
```

```
            )}
          </div>
        </div>
      </div>
    );
  }
```

✅ Step 2: Use this component correctly in `UploadedFilesList.tsx`

File path:

`components/UploadedFilesList.tsx`

Import:

TypeScript

✅ Step 3: Use backend stream URL, not `sharepoint_url` directly

This is the most important part.

❌ Do NOT do this for video:

```
</> TypeScript
```

✅ Do this:

```
<CustomVideoPlayer
  src={`${API}/api/attachments/${file.id}/stream/`}
  fileName={file.file_name}
/>
```

```
</> TypeScript
```

✅ Replace your video block with this exact code

Inside `FileCard`, use this:

```
{file.asset_type === "file" && file.sharepoint_url ? (
  isVideoFile(file) ? (
    <CustomVideoPlayer
      src={`${API}/api/attachments/${file.id}/stream/`}
      fileName={file.file_name}
    />
  ) : (
    <a
      href={file.sharepoint_url}
      target="_blank"
      rel="noopener noreferrer"
      className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark:hover
    >
      View File
```

```
      </a>
    )
  ) : file.asset_type === "url" && file.external_url ? (
    <a
      href={file.external_url}
      target="_blank"
      rel="noopener noreferrer"
      className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark:hover:t
    >
      View Link
    </a>
  ) : (
    <span className="text-xs text-slate-400 dark:text-slate-500">
      No link available
    </span>
  )}
```

🚨 Step 4: Quick test — confirm if problem is URL or player

Before doing backend stream endpoint, do this temporary test:

In any page, temporarily render:

```
<video
  src={file.sharepoint_url || ""}
  controls
  className="w-full"
/>
```

If this also does not play:

👉 Then the issue is **100% URL/backend**, not the player.

✅ Why your current video probably does not play

Because SharePoint usually stores URLs like:

- preview page
- web UI page
- document viewer URL
- protected URL needing cookies/auth

The browser `<video>` element needs:

- raw video bytes
- correct `Content-Type` like `video/mp4`
- ideally range/partial content support

If it gets:

- HTML page
- redirect
- login page
- 302/403/401

Then video won't play.

✅ Proper production solution (must do for working playback)

You need a Django endpoint like:

```bash
```

And then your custom player should use that endpoint.

✅ Recommended backend flow

Your Django endpoint should:

1. Get attachment by `id`
2. Get SharePoint / Graph access token
3. Use stored SharePoint item ID / drive item ID
4. Fetch file from Graph
5. Stream it back to frontend
6. Forward `Range` header (important for seek)
7. Return correct `Content-Type`

🔥 Best next step for your project

Since you're already working with:

- SharePoint upload
- delete APIs
- attachment model

- Django backend

👉 **You should now implement backend streaming**.

Otherwise even the best custom player won't solve it.

✅ What I can do next (best option)

I can now give you the **exact copy-paste backend code** for:

- `attachments/urls.py`
- `attachments/views.py`
- optional model usage
- **working** `/api/attachments/<id>/stream/`
- with **Range header support** (so seek/drag works properly)
- then I'll also give you the **final cleaned** `UploadedFilesList.tsx`

📌 What I need from you now

Send me these 3 backend files from your attachments app:

1. `attachments/models.py`
2. `attachments/views.py`
3. `attachments/urls.py`

And I will give you **properly working exact code** for your project structure — **not generic code**.

This is the correct next move.

Because right now:

- **Frontend player UI = solved**
- **Actual playback issue = backend/SharePoint stream issue**

👉 Send backend files, and I'll make the **real working solution** for you step by step with exact paths.

in uploaded files list i want to show only play video button and after that a popup opens and video is played in it using iframe or such technology now tell me how to do it
step by step with propar path

Yes — **this is a much better approach for your current SharePoint setup** ✅

Since direct inline `<video>` is failing, the **best practical solution right now** is:

- In `UploadedFilesList.tsx`
- For video files:
  - show only a "▶️ **Play Video**" button
- On click:
  - open a **popup / modal**
  - inside popup load the video using:
    - `iframe` (best for SharePoint preview लिंक)
    - OR fallback to `<video>` if you later use backend stream endpoint

👉 Since your SharePoint URL likely opens a preview page, **iframe is the correct choice for now**.

✅ Final approach we will build

Files involved (proper paths)

From your structure:

```
LEADERBOARD_UI/
└── components/
    ├── UploadedFilesList.tsx        <-- update this
    ├── ConfirmDialog.tsx
    ├── Toast.tsx
    └── VideoPreviewModal.tsx        <-- NEW file
```

✅ What will happen

In **UploadedFilesList**:

- PDF/DOC/etc → `View File`
- URL → `View Link`
- Video files (`.mp4`, `.mov`, `.webm`, etc.) → **Play Video button**

When user clicks **Play Video**:

- A modal opens
- Inside modal:

- `iframe` loads `sharepoint_url`
- SharePoint preview page/video opens inside popup

✅ Step 1: Create a new modal component

Proper file path:

`components/VideoPreviewModal.tsx`

Create this file and paste:

```
"use client";

type VideoPreviewModalProps = {
  open: boolean;
  title?: string;
  videoUrl?: string | null;
  onClose: () => void;
};

export default function VideoPreviewModal({
  open,
  title = "Video Preview",
  videoUrl,
  onClose,
}: VideoPreviewModalProps) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center">
```

```
        {/* Backdrop */}
        <div
          className="absolute inset-0 bg-black/70 backdrop-blur-sm"
          onClick={onClose}
        />

        {/* Modal */}
        <div className="relative z-[101] w-[95vw] max-w-5xl h-[85vh] rounded-2xl bg-white dark:bg-s
          {/* Header */}
          <div className="flex items-center justify-between px-4 py-3 border-b border-slate-200 dar
            <div className="min-w-0">
              <h3 className="text-sm font-semibold text-slate-900 dark:text-slate-100 truncate">
                {title}
              </h3>
              <p className="text-xs text-slate-500 dark:text-slate-400">
                Video preview
              </p>
            </div>

            <button
              onClick={onClose}
              className="px-3 py-1.5 text-xs rounded-lg bg-red-600 text-white hover:bg-red-700 tran
            >
              Close
            </button>
          </div>

          {/* Body */}
          <div className="w-full h-[calc(85vh-64px)] bg-black">
            {videoUrl ? (
              <iframe
                src={videoUrl}
                title={title}
                className="w-full h-full"
                allow="autoplay; fullscreen"
```

```
            />
          ) : (
            <div className="w-full h-full flex items-center justify-center text-white text-sm">
              No video URL available
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

✅ Step 2: Update `UploadedFilesList.tsx`

Proper file path:

`components/UploadedFilesList.tsx`

2.1 Add import at top

Add this import:

TypeScript

So your imports become:

```
import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import VideoPreviewModal from "@/components/VideoPreviewModal";
```

✅ Step 3: Add modal state in `UploadedFilesList.tsx`

Inside your component, after existing states, add:

```
const [videoToPreview, setVideoToPreview] = useState<{
  url: string;
  title: string;
```

Put it near your other states like this:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [clients, setClients] = useState<Option[]>([]);
const [draftAssignments, setDraftAssignments] = useState<any>({});
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [filteredPrograms, setFilteredPrograms] = useState<Record<number, Option[]>>({});
const [search, setSearch] = useState("");
```

```
const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
const [videoToPreview, setVideoToPreview] = useState<{
  url: string;
  title: string;
} | null>(null);
const { toasts, showToast, removeToast } = useToast();
```

✅ Step 4: Add helper function to detect video files

Inside `UploadedFilesList.tsx`, add this function **inside the component**, before `const filteredFiles = ...`

```
function isVideoFile(file: Attachment) {
  if (file.asset_type !== "file" || !file.file_name) return false;

  const lower = file.file_name.toLowerCase();

  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v") ||
    lower.endsWith(".avi")
  );
}
```

✅ Step 5: Add function to open video popup

Still inside `UploadedFilesList.tsx`, add this function:

```
function openVideoPreview(file: Attachment) {
  if (!file.sharepoint_url) {
    showToast("Video URL not available", "error");
    return;
  }

  setVideoToPreview({
    url: file.sharepoint_url,
    title: file.file_name,
  });
}
```

✅ Step 6: Replace current file/video render block in `FileCard`

In your `FileCard`, you currently have this section:

```
        {(file.asset_type === "file" && file.sharepoint_url) ||
          (file.asset_type === "url" && file.external_url) ? (
          <a
            href={
              file.asset_type === "file"
                ? file.sharepoint_url!
                : file.external_url!
            }
            target="_blank"
            rel="noopener noreferrer"
```

```
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
          >
            View {file.asset_type === "file" ? "File" : "Link"}
          </a>
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}
```

🔁 Replace that full block with this:

```
        {file.asset_type === "file" && file.sharepoint_url ? (
          isVideoFile(file) ? (
            <button
              onClick={() => openVideoPreview(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5 text-xs rounded-lg bg-em
            >
              ▶ Play Video
            </button>
          ) : (
            <a
              href={file.sharepoint_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
            >
              View File
            </a>
```

```
          )
        ) : file.asset_type === "url" && file.external_url ? (
          <a
            href={file.external_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
          >
            View Link
          </a>
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}
```

✅ Step 7: Add the modal JSX in return section

Inside your main `return (...)` of `UploadedFilesList.tsx`, add the modal **below** `ToastContainer` (or before grid).

Add this:

```
      <VideoPreviewModal
        open={videoToPreview != null}
        title={videoToPreview?.title || "Video Preview"}
        videoUrl={videoToPreview?.url || null}
```

TypeScript

Place it here:

```
<ConfirmDialog
  open={fileIdToDelete != null}
  title="Delete file?"
  description="This action cannot be undone."
  confirmLabel="Delete"
  cancelLabel="Cancel"
  variant="danger"
  onConfirm={confirmDeleteFile}
  onCancel={() => setFileIdToDelete(null)}
/>

<ToastContainer toasts={toasts} onClose={removeToast} />

<VideoPreviewModal
  open={videoToPreview != null}
  title={videoToPreview?.title || "Video Preview"}
  videoUrl={videoToPreview?.url || null}
  onClose={() => setVideoToPreview(null)}
/>
```

✅ Final behavior after these changes

For uploaded files list:

If file is:

- `.mp4`, `.mov`, `.webm`, etc.
  - show ▶️ **Play Video**
  - click → popup opens → `iframe` loads SharePoint preview page
- non-video file
  - show **View File**
- URL asset
  - show **View Link**

🔥 Important note about iframe + SharePoint

This will work **only if SharePoint allows embedding**.

Sometimes SharePoint may block iframe with:

- `X-Frame-Options: SAMEORIGIN`
- `Content-Security-Policy: frame-ancestors`

If iframe gets blocked:

You'll see:

- blank modal
- or browser console error:
  - "Refused to display in a frame..."

👉 If that happens, **iframe cannot be used** for that SharePoint URL.

✅ Best fallback if iframe is blocked

If SharePoint blocks iframe, then instead of embedding, open it in a new tab.

You can still keep the modal UI but add a fallback button.

✅ Better modal version with fallback "Open in New Tab"

If you want a safer version, use this **improved modal** instead.

Replace `components/VideoPreviewModal.tsx` with this better version:

```
"use client";

type VideoPreviewModalProps = {
  open: boolean;
  title?: string;
  videoUrl?: string | null;
  onClose: () => void;
};

export default function VideoPreviewModal({
  open,
  title = "Video Preview",
  videoUrl,
  onClose,
}: VideoPreviewModalProps) {
  if (!open) return null;
```

```
    return (
      <div className="fixed inset-0 z-[100] flex items-center justify-center">
        <div
          className="absolute inset-0 bg-black/70 backdrop-blur-sm"
          onClick={onClose}
        />

        <div className="relative z-[101] w-[95vw] max-w-5xl h-[85vh] rounded-2xl bg-white dark:bg-s
          <div className="flex items-center justify-between px-4 py-3 border-b border-slate-200 dar
            <div className="min-w-0">
              <h3 className="text-sm font-semibold text-slate-900 dark:text-slate-100 truncate">
                {title}
              </h3>
              <p className="text-xs text-slate-500 dark:text-slate-400">
                Video preview
              </p>
            </div>

            <div className="flex items-center gap-2">
              {videoUrl && (
                <a
                  href={videoUrl}
                  target="_blank"
                  rel="noopener noreferrer"
                  className="px-3 py-1.5 text-xs rounded-lg bg-emerald-600 text-white hover:bg-emer
                >
                  Open in New Tab
                </a>
              )}

              <button
                onClick={onClose}
                className="px-3 py-1.5 text-xs rounded-lg bg-red-600 text-white hover:bg-red-700 tr
              >
                Close
```

```
            </button>
          </div>
        </div>

        <div className="w-full h-[calc(85vh-64px)] bg-black">
          {videoUrl ? (
            <iframe
              src={videoUrl}
              title={title}
              className="w-full h-full"
              allow="autoplay; fullscreen"
            />
          ) : (
            <div className="w-full h-full flex items-center justify-center text-white text-sm">
              No video URL available
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

👉 This is better because if iframe fails, user can still click **Open in New Tab**.

✅ Recommended exact order for you

Phase 1 (Quick working UI)

1. Create `components/VideoPreviewModal.tsx`
2. Import it in `components/UploadedFilesList.tsx`

3. Add `videoToPreview` state
4. Add `isVideoFile()` helper
5. Add `openVideoPreview()`
6. Replace file render block with **Play Video**
7. Add modal JSX in return
8. Test with a SharePoint video file

⚠️ If iframe still does not show video

Then SharePoint is blocking iframe.

In that case:

Best production approach:

Use backend endpoint like:

```bash
```

or

```bash
```

Then popup can show:

- `<video controls src="/api/attachments/123/stream/" />`

That is the **real permanent solution**.

✅ My recommendation for your exact case

For **right now**, use:

- **Play Video button**
- **Modal popup**
- **iframe + Open in New Tab fallback**

This is the **fastest and safest** with your current SharePoint URLs.

🚀 If you want, I can now give you the FULL final updated `UploadedFilesList.tsx` file (complete copy-paste version) with:

- `Play Video` button
- `VideoPreviewModal`
- all your current delete/assign logic preserved
- no mistakes in placement

👉 If you want, say:

"give me full final UploadedFilesList.tsx code"

and I'll give you the **exact full file ready to paste**.

VideoPreviewModal.tsx
this is not a valid name give me some other name and all steps correctlly

Yes 👍 good catch.

In **Next.js / TSX**, `VideoPreviewModal.tsx` *should normally be valid*, but if your project/lint/config or naming convention is rejecting it, then we'll use a **simpler, cleaner name**.

✅ Use this file name instead

New component file name:

`VideoPlayerPopup.tsx`

This is short, clear, and safe.

✅ Final correct setup (with proper paths + corrected names)

We will use:

```
LEADERBOARD_UI/
└── components/
```

```bash
```

✅ What we are building

In **UploadedFilesList.tsx**:

- If file is a **video** → show ▶️ **Play Video**
- When clicked:
  - open **popup**
  - inside popup load SharePoint URL using **iframe**
- If non-video file:
  - show **View File**
- If URL:
  - show **View Link**

✅ STEP 1 — Create new popup component

Proper file path:

`components/VideoPlayerPopup.tsx`

Create this file and paste this code

```
"use client";

type VideoPlayerPopupProps = {
  open: boolean;
  title?: string;
  videoUrl?: string | null;
  onClose: () => void;
};

export default function VideoPlayerPopup({
  open,
  title = "Video Player",
  videoUrl,
  onClose,
}: VideoPlayerPopupProps) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center">
      {/* Backdrop */}
      <div
        className="absolute inset-0 bg-black/70 backdrop-blur-sm"
        onClick={onClose}
      />

      {/* Popup container */}
      <div className="relative z-[101] w-[95vw] max-w-5xl h-[85vh] rounded-2xl bg-white dark:bg-s
        {/* Header */}
        <div className="flex items-center justify-between px-4 py-3 border-b border-slate-200 dar
          <div className="min-w-0">
            <h3 className="text-sm font-semibold text-slate-900 dark:text-slate-100 truncate">
              {title}
            </h3>
            <p className="text-xs text-slate-500 dark:text-slate-400">
              Video Preview
```

```
            </p>
          </div>

          <div className="flex items-center gap-2">
            {videoUrl && (
              <a
                href={videoUrl}
                target="_blank"
                rel="noopener noreferrer"
                className="px-3 py-1.5 text-xs rounded-lg bg-emerald-600 text-white hover:bg-emer
              >
                Open in New Tab
              </a>
            )}

            <button
              onClick={onClose}
              className="px-3 py-1.5 text-xs rounded-lg bg-red-600 text-white hover:bg-red-700 tr
            >
              Close
            </button>
          </div>
        </div>

        {/* Body */}
        <div className="w-full h-[calc(85vh-64px)] bg-black">
          {videoUrl ? (
            <iframe
              src={videoUrl}
              title={title}
              className="w-full h-full"
              allow="autoplay; fullscreen"
            />
          ) : (
            <div className="w-full h-full flex items-center justify-center text-white text-sm">
```

```
                No video URL available
              </div>
            )}
          </div>
        </div>
      </div>
    );
  }
```

✅ STEP 2 — Update `UploadedFilesList.tsx` import

Proper file path:

`components/UploadedFilesList.tsx`

At the top of your file, add this import:

TypeScript

So your imports should look like this:

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
```

```
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import VideoPlayerPopup from "@/components/VideoPlayerPopup";
```

✅ STEP 3 — Add popup state

Inside your `UploadedFilesList` component, add this state.

Put it near your other `useState` hooks:

```
const [videoToPreview, setVideoToPreview] = useState<{
  url: string;
  title: string;
```

TypeScript

Your state section should look like this:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [clients, setClients] = useState<Option[]>([]);
const [draftAssignments, setDraftAssignments] = useState<any>({});
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [filteredPrograms, setFilteredPrograms] = useState<Record<number, Option[]>>({});
const [search, setSearch] = useState("");
const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
```

```
const [videoToPreview, setVideoToPreview] = useState<{
  url: string;
  title: string;
} | null>(null);
const { toasts, showToast, removeToast } = useToast();
```

✅ STEP 4 — Add helper function to detect video files

Inside `UploadedFilesList.tsx`, **inside the component**, add this function.

Put it before:

TypeScript

Add this:

```
function isVideoFile(file: Attachment) {
  if (file.asset_type !== "file" || !file.file_name) return false;

  const lower = file.file_name.toLowerCase();

  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v") ||
```

```
      lower.endsWith(".avi")
    );
  }
```

✅ STEP 5 — Add function to open popup

Still inside `UploadedFilesList.tsx`, add this function.

Put it below `isVideoFile()` or before `const filteredFiles = ...`

```
function openVideoPreview(file: Attachment) {
  if (!file.sharepoint_url) {
    showToast("Video URL not available", "error");
    return;
  }

  setVideoToPreview({
    url: file.sharepoint_url,
    title: file.file_name,
  });
}
```

✅ STEP 6 — Replace the "View File / View Link" section in `FileCard`

Inside your `FileCard(file: Attachment)` function, find this current block:

```
{(file.asset_type === "file" && file.sharepoint_url) ||
  (file.asset_type === "url" && file.external_url) ? (
  <a
    href={
      file.asset_type === "file"
        ? file.sharepoint_url!
        : file.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
  >
    View {file.asset_type === "file" ? "File" : "Link"}
  </a>
) : (
  <span className="text-xs text-slate-400 dark:text-slate-500">
    No link available
  </span>
)}
```

❌ Remove that full block

✅ Replace it with this exact code:

```
{file.asset_type === "file" && file.sharepoint_url ? (
  isVideoFile(file) ? (
    <button
```

```
            onClick={() => openVideoPreview(file)}
            className="mt-2 inline-flex items-center gap-2 px-3 py-1.5 text-xs rounded-lg bg-em
          >
            ▶ Play Video
          </button>
        ) : (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
          >
            View File
          </a>
        )
      ) : file.asset_type === "url" && file.external_url ? (
        <a
          href={file.external_url}
          target="_blank"
          rel="noopener noreferrer"
          className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
        >
          View Link
        </a>
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}
```

✅ STEP 7 — Add popup component inside main return

Inside your main `return (...)` in `UploadedFilesList.tsx`, add the popup component.

Best place:

Below `ToastContainer` and above the grid

Find this section:

TypeScript

Right below it, add this:

```
<VideoPlayerPopup
  open={videoToPreview != null}
  title={videoToPreview?.title || "Video Player"}
  videoUrl={videoToPreview?.url || null}
  onClose={() => setVideoToPreview(null)}
/>
```

So it becomes:

```
      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <VideoPlayerPopup
        open={videoToPreview != null}
        title={videoToPreview?.title || "Video Player"}
        videoUrl={videoToPreview?.url || null}
        onClose={() => setVideoToPreview(null)}
      />
```

✅ STEP 8 — Final behavior after changes

Now your Learning Resources uploaded files list will behave like this:

For video files:

- show button:
  - ▶️ **Play Video**
- click button:
  - popup opens
  - SharePoint video/preview page loads inside `iframe`

For normal files:

- show:
  - **View File**

For URL assets:

- show:
  - **View Link**

⚠️ IMPORTANT: If iframe still does not show content

If SharePoint blocks iframe, you may see:

- blank popup
- or browser console error like:
  - **Refused to display in a frame**
  - **X-Frame-Options**
  - **frame-ancestors**

If that happens:

The popup still has **Open in New Tab**, so user can still watch video there.

👉 This is the best temporary solution for SharePoint URLs.

✅ Exact files you need after this

1. New file

`components/VideoPlayerPopup.tsx`

2. Update file

`components/UploadedFilesList.tsx`

🔥 Best recommendation for your project

For **now**:

- use this popup + iframe approach

For **final production** later:

- create backend endpoint like:
  - `/api/attachments/<id>/stream/`
- then inside popup use:
  - `<video controls src="/api/attachments/123/stream/" />`

That will be the real stable solution.

✅ If you want next, I can give you the FULL FINAL COMPLETE `UploadedFilesList.tsx` (entire file, copy-paste ready) with:

- `VideoPlayerPopup`
- video detection
- play button
- popup integration

- your delete + assignment logic untouched

👉 Just say:

“give me full final UploadedFilesList.tsx”

and I’ll give you the **complete exact code ready to paste**.

okay this is not the case now what we have to do is when url is uploaded and when we click on open url it will redirect us to that url but if the url is for any video like youtube video or any other video i want to play that video within leaderboard app using iframe or custome video player so for that tell me what i need to do step by step with propar path

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";

type Option = { id: number; name: string };

type Attachment = {
```

```
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: Option[];
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {

  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [clients, setClients] = useState<Option[]>([]);
  const [draftAssignments, setDraftAssignments] = useState<any>({});
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [filteredPrograms, setFilteredPrograms] =
useState<Record<number, Option[]>>({});
  const [search, setSearch] = useState("");
```

```
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const { toasts, showToast, removeToast } = useToast();

  async function fetchFilesREST() {
    let url = `${API}/api/attachments/`;

    if (clientId || programId) {
      const params = new URLSearchParams();
      if (clientId) params.append("client", String(clientId));
      if (programId) params.append("program", String(programId));
      url += `?${params.toString()}`;
    }

    const res = await fetch(url);
    setFiles(await res.json());
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey, clientId, programId]);

  async function fetchClientsGQL() {
    const query = `query { clients { id name } }`;

    const res = await fetch(`${API}/api/attachments/graphql/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query }),
```

```
    });

    const json = await res.json();
    setClients(json.data.clients);
  }

  useEffect(() => {
    fetchClientsGQL();
  }, []);

  async function fetchProgramsByClientGQL(fileId: number, clientId:
number) {
    const query =
      query($clientId: Int!) {
        programsByClient(clientId: $clientId) {
          id
          name
        }
      }
    ;

    const res = await fetch(`${API}/api/attachments/graphql/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query, variables: { clientId } }),
    });

    const json = await res.json();
```

```
    setFilteredPrograms(prev => ({
      ...prev,
      [fileId]: json.data.programsByClient,
    }));
  }

  async function saveAssignmentREST(fileId: number) {
    const data = draftAssignments[fileId];

    if (!data?.clients?.length || !data?.programs?.length) {
      showToast("Select Client and Program", "error");
      return;
    }

    await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });

    fetchFilesREST();
    // Reset draft selection for this file so dropdowns start blank next
time
    setDraftAssignments((prev: any) => {
      const next = { ...prev };
      delete next[fileId];
      return next;
    });
    setOpenMenu(null);
```

```
}

function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;
  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles(prev => prev.filter(f => f.id !== fileId));
    showToast("File deleted successfully", "success");
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}
```

```
function updateDraft(fileId: number, field: string, values: number[]) {
  setDraftAssignments((prev: any) => ({
    ...prev,
    [fileId]: { ...prev[fileId], [field]: values },
  }));
}

const filteredFiles = files.filter(f =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

const unassigned = filteredFiles.filter(
  f => f.clients.length === 0 && f.programs.length === 0
);

const assigned = filteredFiles.filter(
  f => f.clients.length > 0 || f.programs.length > 0
);

function FileCard(file: Attachment) {

  const cardRef = useRef<HTMLDivElement>(null);
  const draft = draftAssignments[file.id] || {};
  const disableSave = !draft.clients?.length || !draft.programs?.length;

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      const target = event.target as HTMLElement | null;
```

```
        // If click is inside any CustomDropdown overlay, don't close the
assignment popup
        if (target && target.closest(".custom-dropdown-overlay")) {
          return;
        }
        if (
          openMenu === file.id &&
          cardRef.current &&
          !cardRef.current.contains(event.target as Node)
        ) {
          setOpenMenu(null);
        }
      }

      document.addEventListener("mousedown", handleClickOutside);
      return () => {
        document.removeEventListener("mousedown",
handleClickOutside);
      };
    }, [openMenu]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
      >
```

```
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() =>
            setOpenMenu(openMenu === file.id ? null : file.id)
          }
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold">{file.file_name}</p>
        {file.description && <DescriptionDisplay html={file.description}
className="text-xs text-slate-500 dark:text-slate-400" />}

        <p className="text-xs">
          Clients: {file.clients.map(c => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map(p => p.name).join(", ") || "None"}
        </p>

        {(file.asset_type === "file" && file.sharepoint_url) ||
          (file.asset_type === "url" && file.external_url) ? (
          <a
            href={
              file.asset_type === "file"
                ? file.sharepoint_url!
                : file.external_url!
```

```
          }
          target="_blank"
          rel="noopener noreferrer"
          className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
        >
          View {file.asset_type === "file" ? "File" : "Link"}
        </a>
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}

      {openMenu === file.id && (
        <div
          className="
            absolute right-3 top-10
            w-[260px]
            bg-white dark:bg-slate-900
            border border-slate-200 dark:border-slate-700
            p-3 rounded-lg shadow-lg
            space-y-2 z-50
          "
        >

          <CustomDropdown
            value={String(draft.clients?.[0] || "")}
            onChange={(selectedValue) => {
```

```
    const cid = Number(selectedValue);
    updateDraft(file.id, "clients", [cid]);
    updateDraft(file.id, "programs", []);
    fetchProgramsByClientGQL(file.id, cid);
  }}
  options={clients.map((c) => ({ id: String(c.id), label: c.name }))}
  placeholder="Select Client"
  shouldSort={true}
  searchable={clients.length > 5}
  className="w-full text-xs"
/>

<CustomDropdown
  value={String(draft.programs?.[0] || "")}
  onChange={(selectedValue) =>
    updateDraft(file.id, "programs", [Number(selectedValue)])
  }
  options={(filteredPrograms[file.id] || []).map((p) => ({
    id: String(p.id),
    label: p.name,
  }))}
  placeholder="Select Program"
  shouldSort={true}
  className="w-full text-xs"
  searchable={(filteredPrograms[file.id] || []).length > 5}
/>

<div className="flex gap-2 pt-1">
  <button
```

```
              onClick={() => saveAssignmentREST(file.id)}
              disabled={disableSave}
              className="flex-1 px-2 py-1 text-xs rounded bg-emerald-
600 text-white disabled:opacity-50"
            >
              Save
            </button>

            <button
              onClick={() => requestDeleteFile(file.id)}
              className="flex-1 px-2 py-1 text-xs rounded bg-red-600
text-white"
            >
              Delete
            </button>
          </div>
        </div>
      )}
    </div>
  );
}

return (
  <div className="space-y-6">
      <div className="mb-4 flex justify-center">
    <div className="relative w-full max-w-md">
      <svg
        className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
```

```
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
          />
        </svg>
        <input
          type="text"
          placeholder="Search files..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
        />
      </div>
    </div>
      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
```

```
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">

        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100">📂
UnAssigned Files</h3>
          <div className="space-y-3 h-[500px] overflow-y-auto">
            {unassigned.length > 0 ? (
              unassigned.map(file => (
                <FileCard key={file.id} {...file} />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                No unassigned files match your search
              </div>
            )}
          </div>
        </div>

        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
```

```
        <h3 className="mb-3 text-slate-900 dark:text-slate-100">✅ Assigned Files</h3>
        <div className="space-y-3 h-[500px] overflow-y-auto">
          {assigned.length > 0 ? (
            assigned.map(file => (
              <FileCard key={file.id} {...file} />
            ))
          ) : (
            <div className="text-center py-8 text-slate-500 dark:text-slate-400">
              No assigned files match your search
            </div>
          )}
        </div>
      </div>

    </div>
  </div>
 );
}
```

this is my uploadedfileslist.tsx

Perfect — **this is the correct requirement now** ✅

And this is actually **much better and easier** than SharePoint file video playback.

✅ Your exact requirement (final understanding)

For `asset_type === "url"`:

- If URL is a **normal website/article** → keep current behavior:
  - **Open URL** → open in new tab
- If URL is a **video URL** like:
  - YouTube
  - Vimeo
  - Loom
  - direct `.mp4` link
  - other embeddable video links

👉 Then instead of redirecting outside, you want:

- show ▶️ **Play Video**
- click it
- open popup **inside leaderboard app**
- play video using:
  - **iframe** for YouTube/Vimeo/Loom etc.
  - **custom** `<video>` **player** for direct `.mp4/.webm` links

🔥 Best solution architecture

We will do this in **2 frontend files only**:

```
LEADERBOARD_UI/
└── components/
```

Bash

✅ What we will build

In `UploadedFilesList.tsx`

For `asset_type === "url"`:

- Detect if `external_url` is:
  - **YouTube/Vimeo/Loom/embed-able video** → show ▶️ **Play Video**
  - **Direct video file** (`.mp4`, `.webm`, etc.) → show ▶️ **Play Video**
  - otherwise → show **Open URL**

On click:

- open popup modal
- popup decides:
  - **iframe** if embeddable platform URL
  - **custom** `<video controls>` if direct video file URL

✅ STEP 1 — Create new popup component

Proper file path:

`components/UrlVideoPopup.tsx`

Create this file.

Paste this full code:

```
"use client";

type UrlVideoPopupProps = {
  open: boolean;
  title?: string;
  videoUrl?: string | null;
  embedUrl?: string | null;
  isDirectVideo?: boolean;
  onClose: () => void;
};

export default function UrlVideoPopup({
  open,
  title = "Video Player",
  videoUrl,
  embedUrl,
  isDirectVideo = false,
  onClose,
}: UrlVideoPopupProps) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center">
      {/* Backdrop */}
      <div
        className="absolute inset-0 bg-black/70 backdrop-blur-sm"
        onClick={onClose}
      />
```

```
        {/* Popup */}
        <div className="relative z-[101] w-[95vw] max-w-5xl h-[85vh] rounded-2xl bg-white dark:bg-s
          {/* Header */}
          <div className="flex items-center justify-between px-4 py-3 border-b border-slate-200 dar
            <div className="min-w-0">
              <h3 className="text-sm font-semibold text-slate-900 dark:text-slate-100 truncate">
                {title}
              </h3>
              <p className="text-xs text-slate-500 dark:text-slate-400">
                Video Preview
              </p>
            </div>

            <div className="flex items-center gap-2">
              {videoUrl && (
                <a
                  href={videoUrl}
                  target="_blank"
                  rel="noopener noreferrer"
                  className="px-3 py-1.5 text-xs rounded-lg bg-emerald-600 text-white hover:bg-emer
                >
                  Open in New Tab
                </a>
              )}

              <button
                onClick={onClose}
                className="px-3 py-1.5 text-xs rounded-lg bg-red-600 text-white hover:bg-red-700 tr
              >
                Close
              </button>
            </div>
          </div>

          {/* Body */}
```

```
          <div className="w-full h-[calc(85vh-64px)] bg-black flex items-center justify-center">
            {isDirectVideo && videoUrl ? (
              <video
                src={videoUrl}
                controls
                autoPlay
                className="w-full h-full bg-black"
              >
                Your browser does not support video playback.
              </video>
            ) : embedUrl ? (
              <iframe
                src={embedUrl}
                title={title}
                className="w-full h-full"
                allow="autoplay; fullscreen; encrypted-media; picture-in-picture"
                allowFullScreen
              />
            ) : (
              <div className="text-white text-sm text-center px-4">
                Unable to preview this video inside the app.
                <br />
                Please use <span className="font-semibold">Open in New Tab</span>.
              </div>
            )}
          </div>
        </div>
      </div>
    );
  }
```

✅ STEP 2 — Update imports in `UploadedFilesList.tsx`

Proper file path:

`components/UploadedFilesList.tsx`

At the top, add:

TypeScript

So imports become:

```typescript
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";
```

✅ STEP 3 — Add popup state in `UploadedFilesList.tsx`

Inside your component, add this state near your other `useState`s.

Add this:

```
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

Your state section should look like this:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [clients, setClients] = useState<Option[]>([]);
const [draftAssignments, setDraftAssignments] = useState<any>({});
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [filteredPrograms, setFilteredPrograms] = useState<Record<number, Option[]>>({});
const [search, setSearch] = useState("");
const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
const { toasts, showToast, removeToast } = useToast();
```

✅ STEP 4 — Add helper functions in `UploadedFilesList.tsx`

Inside `UploadedFilesList.tsx`, **inside the component**, add these helper functions.

Put them before:

TypeScript

Add this exact block:

```typescript
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();
```

```
    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
    }

    // YouTube normal link: youtube.com/watch?v=VIDEO_ID
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed url
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const shareIndex = pathParts.findIndex((part) => part === "share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
      }

      const embedIndex = pathParts.findIndex((part) => part === "embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }
```

```
    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}
```

✅ STEP 5 — Update the URL render logic inside `FileCard`

Right now you have this block inside `FileCard`:

```
{(file.asset_type === "file" && file.sharepoint_url) ||
  (file.asset_type === "url" && file.external_url) ? (
  <a
    href={
      file.asset_type === "file"
        ? file.sharepoint_url!
        : file.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
  >
    View {file.asset_type === "file" ? "File" : "Link"}
  </a>
) : (
  <span className="text-xs text-slate-400 dark:text-slate-500">
    No link available
  </span>
)}
```

❌ Remove that whole block

✅ Replace with this exact code

```
        {file.asset_type === "file" && file.sharepoint_url ? (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
```

TypeScript

```
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5 text-xs rounded-lg bg-em
            >
              ▶ Play Video
            </button>
          ) : (
            <a
              href={file.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
            >
              Open URL
            </a>
          )
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}
```

✅ STEP 6 — Add popup component in main return

Inside your main `return (...)` of `UploadedFilesList.tsx`, add popup component.

Best place:

Below `ToastContainer`

Find this:

TypeScript

Add this right below it:

```
<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

So it becomes:

```
<ConfirmDialog
  open={fileIdToDelete != null}
  title="Delete file?"
  description="This action cannot be undone."
  confirmLabel="Delete"
  cancelLabel="Cancel"
  variant="danger"
  onConfirm={confirmDeleteFile}
  onCancel={() => setFileIdToDelete(null)}
/>

<ToastContainer toasts={toasts} onClose={removeToast} />

<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

✅ Final behavior after this

For `asset_type === "url"`

If `external_url` is:

1. YouTube

Example:

- `https://www.youtube.com/watch?v=abc123`
- `https://youtu.be/abc123`

👉 Shows:

- ▶️ **Play Video**

👉 On click:

- popup opens
- iframe uses:
  - `https://www.youtube.com/embed/abc123`

---

2. Vimeo

Example:

- `https://vimeo.com/123456`

👉 Shows:

- ▶️ **Play Video**

👉 On click:

- popup opens
- iframe uses:
  - `https://player.vimeo.com/video/123456`

---

3. Loom

Example:

- `https://www.loom.com/share/abc123`

👉 Shows:

- ▶️ **Play Video**

👉 On click:

- popup opens
- iframe uses Loom embed

---

4. Direct video file

Example:

- `https://example.com/video.mp4`

👉 Shows:

- ▶️ **Play Video**

👉 On click:

- popup opens
- `<video controls>` plays inside app

---

5. Normal URL

Example:

- `https://google.com`

- `https://docs.example.com`

👉 Shows:

- **Open URL**
- opens in new tab

⚠️ Important note about "other video URLs"

You said:

> youtube video or any other video

For "**any other**" URL, there are 2 cases:

Case A: Direct file URL

- ends with `.mp4`, `.webm`, etc.
  ✅ Works with `<video>`

Case B: Platform URL (like YouTube/Vimeo/Loom)

✅ Works if we know embed format

Case C: Unknown website with a video page

Example:

- `https://somesite.com/watch/123`

❌ May **not** work unless:

- site allows iframe embedding

- and we know its embed URL format

So for unknown sites:

- safest is still **Open URL**

🔥 Best improvement (recommended later)

If you want this to be more accurate long-term, backend can store an extra field like:

- `url_type: "youtube" | "vimeo" | "loom" | "video_file" | "generic_url"`

Then frontend becomes even cleaner.

But for now, frontend-only solution above is enough.

✅ Exact files to change

1. Create new file:

`components/UrlVideoPopup.tsx`

2. Update existing file:

`components/UploadedFilesList.tsx`

🚀 Best next step for you

Since you already pasted your full file, I can now do the **best thing for you**:

👉 I can give you the **FULL FINAL COMPLETE** `UploadedFilesList.tsx`
(copy-paste ready, with **all your current code preserved** and only video URL popup logic added correctly)

This will be easier for you than manually replacing blocks.

If you want, say exactly:

"give me full final UploadedFilesList.tsx code"

and I'll send you the **entire ready-to-paste file** with:

- `UrlVideoPopup`
- helper functions
- popup state
- updated URL/video logic
- no missing braces / no placement mistakes.

okay now i implemented above steps
"use client";

import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";

type Attachment = {

```
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function DashboardAssets({
  clientId,
  programId,
  embedded,
}: {
  clientId: number | null;
  programId: number | null;
  /** When true, render only inner content (no outer card) for use inside
a parent card */
  embedded?: boolean;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);

  //  Load files when sidebar selection changes
  useEffect(() => {
    if (!clientId || !programId) {
      setFiles([]);
      return;
    }
```

```
    setLoading(true);

    fetch(`${API}/api/attachments/?client=${clientId}&program=${programId}`)
      .then((r) => r.json())
      .then(setFiles)
      .finally(() => setLoading(false));
  }, [clientId, programId]);

  // If nothing selected in sidebar
  if (!clientId || !programId) {
    const fallback = (
      <p className="text-sm text-slate-500 dark:text-slate-400">
        Select client and program from sidebar to view assets.
      </p>
    );
    if (embedded) return fallback;
    return (
      <div className="p-4 border rounded bg-white dark:bg-slate-900">
        {fallback}
      </div>
    );
  }

  // Inner item style matching Program Statistics cards: rounded-lg bg-white border border-slate-100
  const fileItem = (f: Attachment) => (
```

```
    <div key={f.id} className="rounded-lg bg-white p-2.5 sm:p-3 text-sm border border-slate-100 dark:bg-slate-900 dark:border-slate-800 min-w-0">
      <div className="text-xs font-medium text-slate-900 dark:text-slate-50 break-words">{f.file_name}</div>
      {f.description && (
        <DescriptionDisplay html={f.description} className="text-xs text-slate-500 dark:text-slate-400 mt-0.5" />
      )}
      {(f.asset_type === "file" && f.sharepoint_url) ||
      (f.asset_type === "url" && f.external_url) ? (
        <a
          href={
            f.asset_type === "file"
              ? f.sharepoint_url!
              : f.external_url!
          }
          target="_blank"
          rel="noopener noreferrer"
          className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 dark:text-emerald-400 hover:underline"
        >
          View {f.asset_type === "file" ? "file" : "link"}
          <span aria-hidden>→</span>
        </a>
      ) : null}
    </div>
  );
```

```
  const listContent = (
    <>
      {loading && (
        <div className="flex flex-col items-center justify-center py-8
text-center">
          <div className="rounded-full bg-slate-100 dark:bg-slate-800 p-
3 mb-2">
            <FolderOpen className="h-5 w-5 text-slate-400 dark:text-
slate-500 animate-pulse" aria-hidden />
          </div>
          <p className="text-sm text-slate-500 dark:text-slate-
400">Loading files...</p>
        </div>
      )}
      {files.length === 0 && !loading && (
        <div className="flex flex-col items-center justify-center py-8 px-
4 text-center rounded-lg border border-dashed border-slate-200
dark:border-slate-700 bg-slate-50/50 dark:bg-slate-800/30 min-h-
[120px]">
          <FileX2 className="h-8 w-8 text-slate-300 dark:text-slate-600
mb-2" aria-hidden />
          <p className="text-sm font-medium text-slate-600 dark:text-
slate-400">No files assigned</p>
          <p className="text-xs text-slate-500 dark:text-slate-400 mt-
0.5">Assets will appear here when assigned to this program.</p>
        </div>
      )}
      {files.length > 0 && !loading && (
        <div className="grid gap-3">
```

```
          {files.map((f) => fileItem(f))}
        </div>
      )}
    </>
  );

  if (embedded) {
    return (
      <div className="flex flex-col flex-1 min-h-0">
        <div className="flex items-center justify-between flex-shrink-0">
          <div className="inline-flex items-center gap-2 text-xs font-
medium text-slate-600 dark:text-slate-400">
            <FolderOpen className="h-4 w-4 text-emerald-500 flex-
shrink-0" aria-hidden />
            <span className="break-words">Assigned Assets
({files.length})</span>
          </div>
        </div>
        <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
          {listContent}
        </div>
      </div>
    );
  }

  return (
    <section className="mt-6 rounded-xl border border-slate-200
dark:border-slate-700 bg-white dark:bg-slate-900 p-4">
      <h3 className="text-sm font-semibold mb-3 text-slate-900
```

```
dark:text-slate-50">
        Assigned Assets ({files.length})
      </h3>
      {listContent}
    </section>
  );
}
```

this is my dashboard assets do i need to make changes in this too

Yes — **if you want the same "play video inside leaderboard app" behavior in Dashboard Assets too, then you DO need to update this file as well** ✅

Because right now `DashboardAssets.tsx` still does this:

- file → opens new tab
- url → opens new tab

So if a **video URL** is assigned here, it will still redirect outside.

✅ Short answer

Yes, update `DashboardAssets.tsx` too

if you want video URLs to open inside app in dashboard also.

If you want:

- **UploadedFilesList** → play video inside popup

- **DashboardAssets** → also play video inside popup

Then both components need same logic.

🔥 Best approach (recommended)

Since you already created:

- `components/UrlVideoPopup.tsx`

You should **reuse the same popup component** in `DashboardAssets.tsx` too.

That is the cleanest solution.

✅ Proper files involved now

```
LEADERBOARD_UI/
└── components/
    ├── UploadedFilesList.tsx      <-- already updated by you
    ├── DashboardAssets.tsx        <-- YES, update this now
    └── UrlVideoPopup.tsx          <-- reuse this existing file
```

✅ What changes are needed in `DashboardAssets.tsx`

We need to add:

1. Import `UrlVideoPopup`
2. Add popup state
3. Add helper functions:
   - `isDirectVideoUrl`
   - `getEmbedUrl`
   - `isPlayableVideoUrl`
   - `openUrlVideo`
4. Change URL rendering logic:
   - if `asset_type === "url"` and URL is video → show ▶️ **Play Video**
   - else → **Open Link**
5. Add `<UrlVideoPopup />` in return

✅ Important note

For `DashboardAssets.tsx`:

- `asset_type === "file"`
  → keep current `View file` behavior (unless later you want SharePoint file popup too)
- `asset_type === "url"`
  → only for video URLs, show popup

This matches your current requirement exactly.

✅ STEP-BY-STEP changes for `DashboardAssets.tsx`

✅ STEP 1 — Import popup component

File path:

`components/DashboardAssets.tsx`

At top, add:

TypeScript

So imports become:

```typescript
"use client";

import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";
```

✅ STEP 2 — Add popup state

Inside `DashboardAssets`, add this state near other `useState`s.

Add this:

```
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

Your state section should become:

```
const API = process.env.NEXT_PUBLIC_API_BASE_URL;
const [files, setFiles] = useState<Attachment[]>([]);
const [loading, setLoading] = useState(false);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

✅ STEP 3 — Add helper functions

Inside `DashboardAssets`, add these functions **before** `const fileItem = ...`

Add this exact block:

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
```

```
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part === "share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
        }

        const embedIndex = pathParts.findIndex((part) => part === "embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function openUrlVideo(file: Attachment) {
    if (!file.external_url) return;

    const directVideo = isDirectVideoUrl(file.external_url);
    const embedUrl = getEmbedUrl(file.external_url);
```

```
    if (!directVideo && !embedUrl) {
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }
```

✅ STEP 4 — Update the link rendering inside `fileItem`

Right now inside `fileItem`, you have this:

```
{(f.asset_type === "file" && f.sharepoint_url) ||
(f.asset_type === "url" && f.external_url) ? (
  <a
    href={
      f.asset_type === "file"
        ? f.sharepoint_url!
        : f.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 d
  >
    View {f.asset_type === "file" ? "file" : "link"}
```

```
            <span aria-hidden>→</span>
          </a>
        ) : null}
```

❌ Remove that block

✅ Replace with this exact code:

```
        {f.asset_type === "file" && f.sharepoint_url ? (
          <a
            href={f.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 d
          >
            View file
            <span aria-hidden>→</span>
          </a>
        ) : f.asset_type === "url" && f.external_url ? (
          isPlayableVideoUrl(f.external_url) ? (
            <button
              onClick={() => openUrlVideo(f)}
              className="inline-flex items-center gap-1 mt-1.5 px-2.5 py-1 text-xs font-medium roun
            >
              ▶ Play video
            </button>
          ) : (
            <a
              href={f.external_url}
```

```
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600
          >
            Open link
            <span aria-hidden>→</span>
          </a>
        )
      ) : null}
```

✅ STEP 5 — Add popup in both return paths

Because your component has **2 returns**:

1. `if (embedded) return (...)`
2. normal `return (...)`

⚠️ So you must add `UrlVideoPopup` in **both** places.

This is very important.

A better clean solution:

Instead of duplicating popup in both returns, wrap it inside `listContent` or add it in both.

Easiest safe method: add it in both returns.

✅ STEP 6 — Update the `embedded` return

Find this block:

```
if (embedded) {
  return (
    <div className="flex flex-col flex-1 min-h-0">
      <div className="flex items-center justify-between flex-shrink-0">
        <div className="inline-flex items-center gap-2 text-xs font-medium text-slate-600 dark:
          <FolderOpen className="h-4 w-4 text-emerald-500 flex-shrink-0" aria-hidden />
          <span className="break-words">Assigned Assets ({files.length})</span>
        </div>
      </div>
      <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
        {listContent}
      </div>
    </div>
  );
}
```

Replace with this:

```
if (embedded) {
  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
```

```
          title={videoPopup?.title || "Video Player"}
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="flex flex-col flex-1 min-h-0">
          <div className="flex items-center justify-between flex-shrink-0">
            <div className="inline-flex items-center gap-2 text-xs font-medium text-slate-600 dar
              <FolderOpen className="h-4 w-4 text-emerald-500 flex-shrink-0" aria-hidden />
              <span className="break-words">Assigned Assets ({files.length})</span>
            </div>
          </div>
          <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
            {listContent}
          </div>
        </div>
      </>
    );
  }
```

✅ STEP 7 — Update the normal return

Find your final return:

```
  return (
    <section className="mt-6 rounded-xl border border-slate-200 dark:border-slate-700 bg-white da
      <h3 className="text-sm font-semibold mb-3 text-slate-900 dark:text-slate-50">
```

```
        Assigned Assets ({files.length})
      </h3>
      {listContent}
    </section>
  );
```

Replace with this:

```
  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <section className="mt-6 rounded-xl border border-slate-200 dark:border-slate-700 bg-white
        <h3 className="text-sm font-semibold mb-3 text-slate-900 dark:text-slate-50">
          Assigned Assets ({files.length})
        </h3>
        {listContent}
      </section>
    </>
  );
```

✅ Final answer for your question

Do you need to make changes in `DashboardAssets.tsx` too?

✅ YES — if you want the same in-app video behavior here too.

Because currently this component still opens URLs in a new tab.

🔥 Recommended behavior after updating this file

In Dashboard Assets:

- **File asset** (`sharepoint_url`)
  - keep **View file**
- **URL asset**
  - if YouTube/Vimeo/Loom/direct video URL → ▶️ **Play video**
  - otherwise → **Open link**

This is exactly aligned with your new requirement.

🚀 Best next step for you

Since you already pasted the full `DashboardAssets.tsx`, the easiest thing now is:

👉 I can give you the **FULL FINAL COMPLETE** `DashboardAssets.tsx`
(copy-paste ready, with **all changes already applied** and no placement mistakes)

That will save you time and avoid syntax errors.

If you want, say exactly:

“give me full final DashboardAssets.tsx code”

and I’ll send you the **entire ready-to-paste file**.

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState } from "react";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
```

```
const API = process.env.NEXT_PUBLIC_API_BASE_URL;
const [files, setFiles] = useState<Attachment[]>([]);
const [loading, setLoading] = useState(false);

useEffect(() => {
  if (!programId) return;

  async function loadFiles() {
    setLoading(true);

    try {
      const res = await fetch(
        `${API}/api/attachments/?program=${programId}`
      );
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to load program files", err);
    }

    setLoading(false);
  }

  loadFiles();
}, [programId, API]);

if (!programId) return null;

return (
```

```
    <div className="flex flex-col">
      {/* Header (outside the box) */}
      <div className="flex items-center gap-2 mb-3">
        <div className="flex h-8 w-8 items-center justify-center
rounded-lg bg-gradient-to-br from-sky-500 to-sky-600">
          <FolderOpen className="h-4 w-4 text-white" />
        </div>
        <div>
          <h2 className="text-xs lg:text-sm font-semibold text-slate-900
dark:text-slate-50">
            Learning Resources
          </h2>
          <p className="text-[9px] lg:text-xs text-slate-600 dark:text-
slate-400">
            Files & materials
          </p>
        </div>
      </div>

      {/* Content Box */}
      <div className="h-[100px] overflow-y-auto rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/70
p-2.5">
        {loading ? (
          <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-
slate-600" />
```

```
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slate-400">
              Loading files...
            </p>
          </div>
        ) : files.length === 0 ? (
          <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slate-400">
              No files assigned to this program
            </p>
          </div>
        ) : (
          <div className="space-y-2">
            {files.map((file) => (
              <a
                key={file.id}
                href={
                  file.asset_type === "file"
                    ? file.sharepoint_url || "#"
                    : file.external_url || "#"
                }
                target="_blank"
                rel="noopener noreferrer"
                className="flex items-center gap-2 rounded-lg border
```

```
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/60 p-2 hover:bg-slate-50 dark:hover:bg-slate-800 transition-colors"
            >
              <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounded-md bg-slate-100 dark:bg-slate-800 text-slate-600 dark:text-slate-300">
                <FileText className="h-3.5 w-3.5" />
              </div>
              <span className="text-[10px] lg:text-xs font-medium text-emerald-600 truncate">
                {file.file_name}
              </span>
            </a>
          ))}
        </div>
      )}
    </div>
  </div>
 );
}
```

this is my programattachments.tsx i am using this in participant dashboard do i need to make changes here too if yes tell me step by step

Yes — **if you want the same "play video inside app" behavior in Participant Dashboard also, then you need to update `ProgramAttachments.tsx` too** ✅

Right now this component does:

- `file` → opens SharePoint file in new tab
- `url` → opens external URL in new tab

So if a **YouTube/video URL** is assigned here, it will still open outside.

✅ Short answer

YES — update `ProgramAttachments.tsx` too

if you want video URLs in Participant Dashboard to open inside the leaderboard app.

🔥 Best approach (recommended)

Since you already implemented this in other places, **reuse the same popup component**:

- `components/UrlVideoPopup.tsx`

That way all 3 places behave consistently:

1. `UploadedFilesList.tsx`
2. `DashboardAssets.tsx`
3. `ProgramAttachments.tsx` ✅

✅ Files involved now

```
LEADERBOARD_UI/
└── components/
```

Bash

✅ What should happen in `ProgramAttachments.tsx`

For each attachment:

- If `asset_type === "file"`
  - keep current behavior:
  - open SharePoint file in new tab
- If `asset_type === "url"`
  - if URL is:
    - YouTube
    - Vimeo
    - Loom
    - direct `.mp4/.webm/.mov`
  - then:
    - open **inside popup** using `UrlVideoPopup`
  - otherwise:
    - open in new tab

✅ STEP-BY-STEP changes for `ProgramAttachments.tsx`

✅ STEP 1 — Import `UrlVideoPopup`

File path:

`components/ProgramAttachments.tsx`

At the top, add:

TypeScript

So imports become:

```typescript
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";
```

✅ STEP 2 — Add popup state

Inside your component, add this state near the other `useState`s.

Add this:

```
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

Your state section should look like:

```
const API = process.env.NEXT_PUBLIC_API_BASE_URL;
const [files, setFiles] = useState<Attachment[]>([]);
const [loading, setLoading] = useState(false);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

✅ STEP 3 — Add helper functions

Inside `ProgramAttachments.tsx`, add these functions **below the** `useEffect` and **before** `if (!programId) return null;`

Add this exact block:

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
```

```
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part === "share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
        }

        const embedIndex = pathParts.findIndex((part) => part === "embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function handleAttachmentClick(file: Attachment) {
    // FILE assets → keep current behavior
    if (file.asset_type === "file") {
      const fileUrl = file.sharepoint_url;
```

```
      if (fileUrl) {
        window.open(fileUrl, "_blank", "noopener,noreferrer");
      }
      return;
    }

    // URL assets
    const url = file.external_url;
    if (!url) return;

    const directVideo = isDirectVideoUrl(url);
    const embedUrl = getEmbedUrl(url);

    // If not playable video, open normally
    if (!directVideo && !embedUrl) {
      window.open(url, "_blank", "noopener,noreferrer");
      return;
    }

    // Open popup inside app
    setVideoPopup({
      title: file.file_name,
      videoUrl: url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }
```

✅ STEP 4 — Replace clickable `<a>` with `<button>`

Right now in your `files.map(...)`, you are rendering this:

```
{files.map((file) => (
  <a
    key={file.id}
    href={
      file.asset_type === "file"
        ? file.sharepoint_url || "#"
        : file.external_url || "#"
    }
    target="_blank"
    rel="noopener noreferrer"
    className="flex items-center gap-2 rounded-lg border border-slate-200 dark:border
  >
    <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounded-md
      <FileText className="h-3.5 w-3.5" />
    </div>
    <span className="text-[10px] lg:text-xs font-medium text-emerald-600 truncate">
      {file.file_name}
    </span>
  </a>
))}
```

❌ Remove that `<a>...</a>`

✅ Replace with this exact code:

```
{files.map((file) => {
  const isVideoUrl =
```

```
                file.asset_type === "url" &&
                !!file.external_url &&
                isPlayableVideoUrl(file.external_url);

              return (
                <button
                  key={file.id}
                  type="button"
                  onClick={() => handleAttachmentClick(file)}
                  className="w-full flex items-center gap-2 rounded-lg border border-slate-200 da
                >
                  <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounded-
                    <FileText className="h-3.5 w-3.5" />
                  </div>

                  <div className="min-w-0 flex-1">
                    <span className="block text-[10px] lg:text-xs font-medium text-emerald-600 tr
                      {file.file_name}
                    </span>

                    <span className="block text-[9px] lg:text-[11px] text-slate-500 dark:text-sla
                      {file.asset_type === "file"
                        ? "Open file"
                        : isVideoUrl
                        ? "Play video"
                        : "Open link"}
                    </span>
                  </div>
                </button>
              );
            })}
```

✅ STEP 5 — Add popup before main return content

Because this component has only **one main return**, it's easy.

Find this:

```
return (
```

TypeScript

Replace the whole return with this structure:

```
return (
  <>
    <UrlVideoPopup
      open={videoPopup != null}
      title={videoPopup?.title || "Video Player"}
      videoUrl={videoPopup?.videoUrl || null}
      embedUrl={videoPopup?.embedUrl || null}
      isDirectVideo={videoPopup?.isDirectVideo || false}
      onClose={() => setVideoPopup(null)}
    />

    <div className="flex flex-col">
      {/* Header (outside the box) */}
      <div className="flex items-center gap-2 mb-3">
        <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-gradient-to-br f
          <FolderOpen className="h-4 w-4 text-white" />
```

```
            </div>
            <div>
              <h2 className="text-xs lg:text-sm font-semibold text-slate-900 dark:text-slate-50">
                Learning Resources
              </h2>
              <p className="text-[9px] lg:text-xs text-slate-600 dark:text-slate-400">
                Files & materials
              </p>
            </div>
          </div>

          {/* Content Box */}
          <div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-
            {loading ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                  Loading files...
                </p>
              </div>
            ) : files.length === 0 ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                  No files assigned to this program
                </p>
              </div>
            ) : (
              <div className="space-y-2">
                {files.map((file) => {
                  const isVideoUrl =
                    file.asset_type === "url" &&
                    !!file.external_url &&
                    isPlayableVideoUrl(file.external_url);
```

```
                return (
                  <button
                    key={file.id}
                    type="button"
                    onClick={() => handleAttachmentClick(file)}
                    className="w-full flex items-center gap-2 rounded-lg border border-slate-200
                  >
                    <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounde
                      <FileText className="h-3.5 w-3.5" />
                    </div>

                    <div className="min-w-0 flex-1">
                      <span className="block text-[10px] lg:text-xs font-medium text-emerald-600
                        {file.file_name}
                      </span>

                      <span className="block text-[9px] lg:text-[11px] text-slate-500 dark:text-s
                        {file.asset_type === "file"
                          ? "Open file"
                          : isVideoUrl
                          ? "Play video"
                          : "Open link"}
                      </span>
                    </div>
                  </button>
                );
              })}
            </div>
          )}
        </div>
      </div>
    </>
  );
```

✅ Final behavior after these changes

In Participant Dashboard → Learning Resources

If attachment is:

1. File asset

- Opens SharePoint file in new tab
- same as before

2. URL asset = YouTube/Vimeo/Loom/direct `.mp4`

- Opens **inside leaderboard app**
- popup opens using `UrlVideoPopup`

3. URL asset = normal website/article

- Opens in new tab

⚠️ Important note

This will work well for:

- YouTube
- Vimeo
- Loom
- direct `.mp4/.webm/.mov`

For **unknown websites with video pages**, it may still open in new tab unless we specifically support that platform.

That is correct and safest.

✅ So to your exact question:

Do I need to make changes here too?

✅ YES — if you want the same in-app video playback in Participant Dashboard.

🚀 Best next step

Since you already pasted the full `ProgramAttachments.tsx`, the easiest and safest thing now is:

👉 I can give you the **FULL FINAL COMPLETE** `ProgramAttachments.tsx`
(copy-paste ready, with all changes already applied)

That will avoid syntax mistakes.

If you want, say exactly:

"give me full final ProgramAttachments.tsx code"

and I'll send the **entire ready-to-paste file**.

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
```

```
} | null>(null);

useEffect(() => {
  if (!programId) return;

  async function loadFiles() {
    setLoading(true);

    try {
      const res = await fetch(
        ${API}/api/attachments/?program=${programId}
      );
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to load program files", err);
    }

    setLoading(false);
  }

  loadFiles();
}, [programId, API]);

function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
```

```
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
      }

      // YouTube normal link
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        if (parsed.pathname.includes("/embed/")) return url;
      }

      // Vimeo
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
```

```
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? https://player.vimeo.com/video/${videoId} :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}
```

```
function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function handleAttachmentClick(file: Attachment) {
  // FILE assets → keep current behavior
  if (file.asset_type === "file") {
    const fileUrl = file.sharepoint_url;
    if (fileUrl) {
      window.open(fileUrl, "_blank", "noopener,noreferrer");
    }
    return;
  }

  // URL assets
  const url = file.external_url;
  if (!url) return;

  const directVideo = isDirectVideoUrl(url);
  const embedUrl = getEmbedUrl(url);

  // If not playable video, open normally
  if (!directVideo && !embedUrl) {
    window.open(url, "_blank", "noopener,noreferrer");
    return;
  }

  // Open popup inside app
  setVideoPopup({
```

```
      title: file.file_name,
      videoUrl: url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }

  if (!programId) return null;

  return (
    <div className="flex flex-col">
      {/* Header (outside the box) */}
      <div className="flex items-center gap-2 mb-3">
        <div className="flex h-8 w-8 items-center justify-center
rounded-lg bg-gradient-to-br from-sky-500 to-sky-600">
          <FolderOpen className="h-4 w-4 text-white" />
        </div>
        <div>
          <h2 className="text-xs lg:text-sm font-semibold text-slate-900
dark:text-slate-50">
            Learning Resources
          </h2>
          <p className="text-[9px] lg:text-xs text-slate-600 dark:text-
slate-400">
            Files & materials
          </p>
        </div>
      </div>
```

```
      {/* Content Box */}
      <div className="h-[100px] overflow-y-auto rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/70
p-2.5">
        {loading ? (
          <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-
slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
              Loading files...
            </p>
          </div>
        ) : files.length === 0 ? (
          <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-
slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
              No files assigned to this program
            </p>
          </div>
        ) : (
          <div className="space-y-2">
            {files.map((file) => {
```

```
            const isVideoUrl =
              file.asset_type === "url" &&
              !!file.external_url &&
              isPlayableVideoUrl(file.external_url);

            return (
              <button
                key={file.id}
                type="button"
                onClick={() => handleAttachmentClick(file)}
                className="w-full flex items-center gap-2 rounded-lg
border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-
900/60 p-2 hover:bg-slate-50 dark:hover:bg-slate-800 transition-colors
text-left"
              >
                <div className="flex h-7 w-7 flex-shrink-0 items-center
justify-center rounded-md bg-slate-100 dark:bg-slate-800 text-slate-
600 dark:text-slate-300">
                  <FileText className="h-3.5 w-3.5" />
                </div>

                <div className="min-w-0 flex-1">
                  <span className="block text-[10px] lg:text-xs font-
medium text-emerald-600 truncate">
                    {file.file_name}
                  </span>

                  <span className="block text-[9px] lg:text-[11px] text-
slate-500 dark:text-slate-400">
```

```
                    {file.asset_type === "file"
                      ? "Open file"
                      : isVideoUrl
                      ? "Play video"
                      : "Open link"}
                  </span>
                </div>
              </button>
            );
          })}
        </div>
      )}
    </div>
  </div>
);
}
```

can you give me fully updated code file

Yes — your current file is **almost correct**, but you are **missing the** `UrlVideoPopup` **render inside return**.
Without that, clicking **Play video** will set state, but **popup will never open**.

👉 Below is the **FULL FINAL COMPLETE** `ProgramAttachments.tsx` (copy-paste ready).

✅ Full final file

File path:

`components/ProgramAttachments.tsx`

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  useEffect(() => {
```

```
    if (!programId) {
      setFiles([]);
      return;
    }

    async function loadFiles() {
      setLoading(true);

      try {
        const res = await fetch(`${API}/api/attachments/?program=${programId}`);
        const data = await res.json();
        setFiles(data);
      } catch (err) {
        console.error("Failed to load program files", err);
        setFiles([]);
      } finally {
        setLoading(false);
      }
    }

    loadFiles();
  }, [programId, API]);

  function isDirectVideoUrl(url: string) {
    const lower = url.toLowerCase();
    return (
      lower.endsWith(".mp4") ||
      lower.endsWith(".webm") ||
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
```

```
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link: youtu.be/VIDEO_ID
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
      }

      // YouTube normal link: youtube.com/watch?v=VIDEO_ID
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        // already embed URL
        if (parsed.pathname.includes("/embed/")) return url;
      }

      // Vimeo
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part === "share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
        }
```

```
        const embedIndex = pathParts.findIndex((part) => part === "embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function handleAttachmentClick(file: Attachment) {
    // FILE assets -> keep current behavior
    if (file.asset_type === "file") {
      const fileUrl = file.sharepoint_url;
      if (fileUrl) {
        window.open(fileUrl, "_blank", "noopener,noreferrer");
      }
      return;
    }

    // URL assets
    const url = file.external_url;
    if (!url) return;

    const directVideo = isDirectVideoUrl(url);
    const embedUrl = getEmbedUrl(url);

    // If not playable video, open normally
    if (!directVideo && !embedUrl) {
```

```
        window.open(url, "_blank", "noopener,noreferrer");
        return;
      }

      // Open popup inside app
      setVideoPopup({
        title: file.file_name,
        videoUrl: url,
        embedUrl,
        isDirectVideo: directVideo,
      });
    }

    if (!programId) return null;

    return (
      <>
        <UrlVideoPopup
          open={videoPopup != null}
          title={videoPopup?.title || "Video Player"}
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="flex flex-col">
          {/* Header (outside the box) */}
          <div className="flex items-center gap-2 mb-3">
            <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-gradient-to-br f
              <FolderOpen className="h-4 w-4 text-white" />
            </div>
            <div>
              <h2 className="text-xs lg:text-sm font-semibold text-slate-900 dark:text-slate-50">
                Learning Resources
```

```
            </h2>
            <p className="text-[9px] lg:text-xs text-slate-600 dark:text-slate-400">
              Files & materials
            </p>
          </div>
        </div>

        {/* Content Box */}
        <div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-
          {loading ? (
            <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
              <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
              <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                Loading files...
              </p>
            </div>
          ) : files.length === 0 ? (
            <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
              <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
              <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                No files assigned to this program
              </p>
            </div>
          ) : (
            <div className="space-y-2">
              {files.map((file) => {
                const isVideoUrl =
                  file.asset_type === "url" &&
                  !!file.external_url &&
                  isPlayableVideoUrl(file.external_url);

                return (
                  <button
                    key={file.id}
                    type="button"
```

```
                    onClick={() => handleAttachmentClick(file)}
                    className="w-full flex items-center gap-2 rounded-lg border border-slate-200
                  >
                    <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounde
                      <FileText className="h-3.5 w-3.5" />
                    </div>

                    <div className="min-w-0 flex-1">
                      <span className="block text-[10px] lg:text-xs font-medium text-emerald-600
                        {file.file_name}
                      </span>

                      <span className="block text-[9px] lg:text-[11px] text-slate-500 dark:text-s
                        {file.asset_type === "file"
                          ? "Open file"
                          : isVideoUrl
                          ? "Play video"
                          : "Open link"}
                      </span>
                    </div>
                  </button>
                );
              })}
            </div>
          )}
        </div>
      </div>
    </>
  );
}
```

✅ What was missing in your version?

You had everything almost correct, but **this part was missing**:

```
<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

Without this:

- `setVideoPopup(...)` runs
- state changes
- **but popup never renders**

✅ Final behavior now

In Participant Dashboard → Learning Resources

- **File attachment**
  - Opens SharePoint file in new tab
- **URL attachment = YouTube/Vimeo/Loom/direct** `.mp4`
  - Opens **inside leaderboard app popup**
- **Normal URL**

- Opens in new tab

🔥 Important next check

Now test with these 3 URL types:

Test 1 — YouTube

```
https://www.youtube.com/watch?v=dQw4w9WgXcQ
```

Test 2 — YouTube short

```
https://youtu.be/dQw4w9WgXcQ
```

Test 3 — Direct video file

```
https://www.w3schools.com/html/mov_bbb.mp4
```

If these work, your popup logic is correct.

⚠️ One small recommendation

If later you want the **same logic in all 3 places** (`UploadedFilesList`, `DashboardAssets`, `ProgramAttachments`) without repeating helper functions, we should create a shared utility file like:

```bash
```

and move:

- `isDirectVideoUrl`
- `getEmbedUrl`
- `isPlayableVideoUrl`

there.

That would make your code cleaner.

🚀 Best next step for you

If you want, I can now give you the **shared utility file version** so you stop duplicating the same video URL logic in 3 components.

I can give you:

- `lib/videoUrlUtils.ts`
- updated imports for:
  - `UploadedFilesList.tsx`
  - `DashboardAssets.tsx`
  - `ProgramAttachments.tsx`

👉 If you want, say:

“give me shared videoUrlUtils.ts and updated imports”

and I'll give you the clean reusable version.

here i want same play video button which we used in previously

Yes — got it ✅

In `ProgramAttachments.tsx`, you want the **same UI as before**:

- For **video URLs** → show a **green "▶️ Play video" button**
- For **normal URLs** → show **Open link**
- For **file assets** → show **View file**
- And when **Play video** is clicked → open `UrlVideoPopup`

👉 Right now your current `ProgramAttachments.tsx` makes the **whole card clickable**.
You want the **same button-style behavior** like previous components instead.

✅ What needs to change

In `ProgramAttachments.tsx`:

Current behavior:

- whole row is a `<button>`

Required behavior:

- keep row/card layout
- inside row:
  - show file name
  - show action button/link below:
    - `▶ Play video`
    - `Open link`
    - `View file`

✅ Proper file path

Update this file:

```bash
```

✅ Full final updated code (same Play Video button style)

Copy-paste this full file:

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
```

```
import { useEffect, useState } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  useEffect(() => {
    if (!programId) {
      setFiles([]);
      return;
    }

    async function loadFiles() {
      setLoading(true);
```

```
    try {
      const res = await fetch(`${API}/api/attachments/?program=${programId}`);
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to load program files", err);
      setFiles([]);
    } finally {
      setLoading(false);
    }
  }

  loadFiles();
}, [programId, API]);

function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
```

```
    return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
  }

  // YouTube normal link
  if (host.includes("youtube.com")) {
    const videoId = parsed.searchParams.get("v");
    if (videoId) return `https://www.youtube.com/embed/${videoId}`;

    // already embed URL
    if (parsed.pathname.includes("/embed/")) return url;
  }

  // Vimeo
  if (host.includes("vimeo.com")) {
    const pathParts = parsed.pathname.split("/").filter(Boolean);
    const videoId = pathParts[pathParts.length - 1];
    return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
  }

  // Loom
  if (host.includes("loom.com")) {
    const pathParts = parsed.pathname.split("/").filter(Boolean);

    const shareIndex = pathParts.findIndex((part) => part === "share");
    if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
      return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
    }

    const embedIndex = pathParts.findIndex((part) => part === "embed");
    if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
      return url;
    }
  }

  return null;
```

```
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function openUrlVideo(file: Attachment) {
    if (!file.external_url) return;

    const directVideo = isDirectVideoUrl(file.external_url);
    const embedUrl = getEmbedUrl(file.external_url);

    if (!directVideo && !embedUrl) {
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }

  if (!programId) return null;

  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
```

```
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="flex flex-col">
          {/* Header */}
          <div className="flex items-center gap-2 mb-3">
            <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-gradient-to-br f
              <FolderOpen className="h-4 w-4 text-white" />
            </div>
            <div>
              <h2 className="text-xs lg:text-sm font-semibold text-slate-900 dark:text-slate-50">
                Learning Resources
              </h2>
              <p className="text-[9px] lg:text-xs text-slate-600 dark:text-slate-400">
                Files & materials
              </p>
            </div>
          </div>

          {/* Content Box */}
          <div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-
            {loading ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                  Loading files...
                </p>
              </div>
            ) : files.length === 0 ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
```

```
                No files assigned to this program
              </p>
            </div>
          ) : (
            <div className="space-y-2">
              {files.map((file) => {
                const isVideoUrl =
                  file.asset_type === "url" &&
                  !!file.external_url &&
                  isPlayableVideoUrl(file.external_url);

                return (
                  <div
                    key={file.id}
                    className="rounded-lg border border-slate-200 dark:border-slate-800 bg-white
                  >
                    <div className="flex items-start gap-2">
                      <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center roun
                        <FileText className="h-3.5 w-3.5" />
                      </div>

                      <div className="min-w-0 flex-1">
                        <span className="block text-[10px] lg:text-xs font-medium text-slate-900
                          {file.file_name}
                        </span>

                        {/* Action */}
                        <div className="mt-1.5">
                          {file.asset_type === "file" && file.sharepoint_url ? (
                            <a
                              href={file.sharepoint_url}
                              target="_blank"
                              rel="noopener noreferrer"
                              className="inline-flex items-center gap-1 text-[10px] lg:text-xs fo
                            >
```

```
                              View file
                              <span aria-hidden>→</span>
                            </a>
                          ) : file.asset_type === "url" && file.external_url ? (
                            isVideoUrl ? (
                              <button
                                type="button"
                                onClick={() => openUrlVideo(file)}
                                className="inline-flex items-center gap-1 px-2.5 py-1 text-[10px]
                              >
                                ▶ Play video
                              </button>
                            ) : (
                              <a
                                href={file.external_url}
                                target="_blank"
                                rel="noopener noreferrer"
                                className="inline-flex items-center gap-1 text-[10px] lg:text-xs
                              >
                                Open link
                                <span aligned-hidden>→</span>
                              </a>
                            )
                          ) : (
                            <span className="text-[10px] lg:text-xs text-slate-400 dark:text-slat
                              No link available
                            </span>
                          )}
                        </div>
                      </div>
                    </div>
                  </div>
                );
              })}
            </div>
```

```
            )}
          </div>
        </div>
      </>
    );
  }
```

✅ What changed vs your current version

I changed from:

- whole row clickable button

To:

- row is just a card
- inside it:
  - file name
  - **same action button style** as previous components

✅ Final behavior now

For each learning resource in participant dashboard:

File asset

- **View file** link

URL asset (video URL like YouTube/Vimeo/Loom/direct `.mp4`)

- ▶️ **Play video** button
- opens inside `UrlVideoPopup`

URL asset (normal website)

- **Open link**

🔥 My recommendation

Now you have **same behavior across screens**, which is correct:

- `UploadedFilesList.tsx`
- `DashboardAssets.tsx`
- `ProgramAttachments.tsx`

If you want, next I can give you **one common reusable helper file** so you stop duplicating:

- `isDirectVideoUrl`
- `getEmbedUrl`
- `isPlayableVideoUrl`

across all 3 components.

👉 If you want, say:

"give me reusable video utils file now"

and I'll give you:

- `lib/videoUrlUtils.ts`

- exact import changes for all 3 files.